

DMTDM 用户手册



达梦时序数据库

达梦时序数据库简介

- 产品介绍
- 硬件环境需求
- 软件环境需求

安装部署

- 安装前准备工作
 - 检查系统信息
 - 检查操作系统信息
 - 检查防火墙状态
 - 检查python版本及依赖库
- 安装包解压
- 安装更新许可证
- 安装包目录及配置文件介绍
- 单节点模式部署
 - 配置安装
 - 启停和登录
- 分布式集群部署
 - 分布式集群概述
 - 分布式搭建
 - 节点规划
 - 配置安装分布式集群
 - 不同架构搭建分布式
 - 启停和登录
 - 集群扩缩容
 - 集群扩容
 - 集群缩容
- 部署后实例目录说明
- TDM版本查看
- 更新或修改数据库配置
- 卸载

SQL语言简介

- 标识符和关键字
- 常量
 - 字符串常量
 - 数字常量
- 表达式
- 数据类型
 - 数值类型
 - 字符类型
 - 日期时间类型
 - 布尔类型

数据定义

- 管理数据库
 - 创建数据库
 - 使用数据库
 - 查看数据库
- 管理表
 - 表定义语句
 - 定义普通表
 - 定义时序表
 - 修改表
 - 增加列
 - 移除列
 - 更改列的默认值
 - 删除列的默认值
 - 修改列的数据类型
 - 重命名列
 - 重命名表

- 查看表
- 查看表结构
- 删除表
- 模式

数据操纵

- 导入数据
- 数据插入
- LineProtocol协议插入数据
- 数据更新
- 数据删除

数据查询

- FROM子句
- WHERE 子句
- GROUP BY 子句
- HAVING子句
- ORDER BY子句
- SELECT列表
- LIMIT子句
- 子查询
- 组合查询
- 连接查询
- 窗口查询

视图

- 创建视图
- 删除视图

函数和操作符

- 逻辑操作符
- 比较函数和操作符
 - 比较操作符
 - 比较谓词
- 数学操作符和函数
- 字符串函数和操作符
- 模式匹配
- 数据类型格式化函数
- 时间/日期函数和操作符
 - date_bin
 - date_trunc
- 条件表达式
- 聚集函数
- 窗口函数
- 类型转换函数
- 行比较
 - IN
 - NOT IN
- 时序函数
 - time_bucket
 - days_in_month
 - month_normalize
 - first
 - last

结果集缓存

- 建表设置缓存
- 修改表缓存
- 查看表缓存

发布订阅功能

- 创建主题
- 删除主题
- 订阅主题并消费

地理数据类型支持

OGC Geometry

WKT和EWKT

Point

LineString

Polygon

ST_GeomFromText

ST_Polygon

ST_DWithin

ST_Contains

geometry数据类型及空间函数使用实例

预编译语句

PREPARE

EXECUTE

DEALLOCATE

备份与恢复

备份

恢复

应用程序连接示例

JAVA

JDBC简介

使用前准备

应用示例

C/C++

使用前准备

应用示例

C#

使用前准备

应用示例

Python

使用前准备

应用示例

Go

使用前准备

应用示例

时间分区和滚动删除

分区查询

自定义时间分区

滚动删除

数据保留

创建保留策略

移除保留策略

查询保留策略

数据库角色

基本概念

角色

角色属性

角色组

权限分类

创建角色

删除角色

修改角色

修改角色属性

修改角色名

修改角色组

赋予权限

赋予数据库对象权限

赋予成员关系

回收权限

回收数据库对象权限

回收成员关系

服务器配置

配置文件位置

连接设置

传输加密SSL

存储加密

结果集缓存

附录1 关键字

附录2 SQL语法描述说明

附录3 达梦时序数据库技术支持

达梦时序数据库简介

产品介绍

达梦时序数据库（以下简称TDM）是由达梦数据自主研发的深度兼容TimescaleDB的分布式时序数据库，提供批流一体的数据实时处理引擎，兼容传统SQL、提供数据订阅、预警等内置引擎，可以有效支撑物联网和工业互联网、数字孪生、金融交易、业务监控和系统运维领域等多种时序数据业务场景。达梦时序数据库支持国内外主流平台，例如UOS，Red hat、centos、麒麟等操作系统，x86、AARCH64、MIPS等cpu架构。

硬件环境需求

达梦时序数据库支持各种主流的软硬件环境，用户应根据应用系统的需求来选择合适的软硬件配置，如CPU的指标、内存及磁盘容量等。下面给出安装达梦时序数据库所需的硬件基本配置：

表1.1 硬件环境需求

名称	要求
CPU	2.0GHz 4核心 64位处理器
内存	16G（建议32G及以上）DDR4
硬盘	5G以上存储空间、建议使用SSD
网卡	1000M以上支持TCP/IP协议的网卡
光驱	32倍速以上光驱

软件环境需求

运行达梦时序数据库所要求的软件环境主要有：

表1.2 软件环境需求

名称	要求
操作系统	Linux（建议内核3.15 版本及以上）
网络协议	TCP/IP协议
系统盘	至少1G以上的剩余空间
其他软件	Python3.7

安装部署

安装前准备工作

用户在安装TDM之前需要对部署目标服务器检查或修改操作系统的配置，确认TDM安装包和当前系统匹配，以保证TDM能够正确安装和正常运行。

检查系统信息

用户在安装TDM前需要检查操作系统的相关信息，确认TDM和当前架构及系统匹配，以保证TDM能够正确安装和正常运行；确认内存和磁盘等硬件资源满足业务场景、性能要求。

1. 查看当前的cpu架构及相关信息。

```
#获取cpu信息
```

```
lscpu
```

2. 查看当前操作系统的相关信息。

```
#获取系统位数
```

```
getconf LONG_BIT
```

```
#查询操作系统 release 信息
```

```
cat /etc/os-release
```

```
#查询系统内核信息
```

```
uname -a
```

3. 查看内存信息

```
#获取内存总大小
```

```
grep MemTotal /proc/meminfo
```

```
#获取交换分区大小
```

```
grep SwapTotal/proc/meminfo
```

```
#获取内存使用详情（分别以KB、MB、GB为单位显示，根据需要）
```

```
free
```

```
free -m
```

```
free -g
```

4. 查看存储空间、根据磁盘目录结构和剩余空间规划数据路径：

```
#检查磁盘分区及挂载点
```

```
df -h
```

```
#查看详细分区信息
```

```
lsblk
```

检查操作系统信息

用户在安装TDM之前需要检查ulimit的参数，并根据实际业务量设置对应的open files参数，以保证TDM能够正常运行。

1. 检查当前安装用户的ulimit参数：

```
#检查所有参数限制
```

```
ulimit -a
```

```
#检查当前用户的进程打开文件数限制（软限制）
```

```
ulimit -n
```

open files（-n）建议用户设置65536以上或者unlimited（无限制）；TDM对open files的参数设置要求和业务量相关，表的数量和数据量越多，open files的值也越大

2. 修改当前安装用户的open files参数：

```
#临时修改
```

```
ulimit -n 65536
```

永久修改（需root权限）用户级限制

```
sudo vim /etc/security/limits.conf
```

在文件末尾添加以下内容（以用户username为例）soft 是软限制，hard 是硬限制

```
username soft nofile 65536
```

```
username hard nofile 65536
```

设置后再次查看参数限制

```
ulimit -a
```

3. 部署启动过程需要的系统命令检查：

```
#tar命令检查
```

```
tar --help
```



```
#hostname命令检查
```

```
hostname -I
```

```
#addr2line命令检查
```

```
addr2line --help
```

```
#addr2line命令安装
```

```
sudo yum install binutils
```

或者离线环境需要下载对应的rpm包（binutils-2.41-*.rpm binutils-gold-*.rpm）安装

```
rpm -ivh binutils-2.41-5.oe2403.x86_64.rpm binutils-gold-*.rpm
```

检查防火墙状态

TDM的正常运行使用，需要防火墙关闭或者添加白名单。

1. 检查防火墙运行状态及关闭命令：

```
#查看防火墙状态
```

```
systemctl status firewalld
```

```
#关闭防火墙（重启后恢复）
```

```
systemctl stop firewalld
```

```
#永久关闭（禁止开机启动）
```

```
systemctl disable firewalld
```

检查python版本及依赖库

TDM 的安装脚本与启停脚本均基于 Python 环境开发，执行安装部署前，请确认系统 Python 版本为 3.7。若采用分布式部署，需额外安装 paramiko 依赖库。

检查python版本：

```
python --version
```

```
python3 --version
```

检查python依赖库：

```
python3 -m pip list
```

Python3.7安装

在线安装

```
yum install python37
```

离线安装

1.解压源码包

```
tar -xvf Python-3.7.10.tgz  
  
cd Python-3.7.10/
```

2.配置安装选项

```
./configure --enable-optimizations --prefix=/usr/local/Python3.7
```

3.编译安装

```
make -j$(nproc) # 使用所有CPU核心加速编译  
  
sudo make install
```

4.验证安装

```
python3 --version
```

5.TDM使用python3.7

方案1：临时将系统python3 的软连接改为指向3.7

方案2：显式地调用python3.7,例如:

```
/usr/local/python3.7/bin/python3.7 执行脚本
```

Paramiko安装:

1. 在线安装

```
yum install -y python3-paramiko
```

2. 离线安装

首先在联网的机器上下载 paramiko 及其所有依赖

```
创建存放目录  
mkdir -p /root/paramiko_rpms && cd /root/paramiko_rpms  
  
下载依赖库  
Yumdownloader --destdir=/root/paramiko_rpms python3-paramiko python3-bcrypt python3-  
pynacl python3-cryptography python3-six python3-pyasn1 libsodium python3-cffi python3-  
pyparser libffi  
  
打包上述依赖库  
tar -czvf paramiko_rpms.tar.gz /root/paramiko_rpms
```

将下载的包上传到离线的安装环境并安装

```
tar -xzvf paramiko_rpms.tar.gz
```

```
cd /path/paramiko_rpms
```

安装基础依赖库及paramiko（按顺序）

```
sudo rpm -ivh libffi-*.rpm libsodium-*.rpm python3-setuptools-*.rpm python3-six*.rpm  
python3-pycparser*.rpm python3-cffi*.rpm python3-cryptography*.rpm python3-bcrypt-*.rpm  
python3-pynacl-*.rpm python3-pyasn1-*.rpm python3-paramiko*.rpm
```

3. 验证安装有版本输出

```
python3 -c "import paramiko; print(paramiko.__version__)"
```

安装包解压

本产品提供RPM、DEB 及 TAR三种格式的安装包，本质均为文件压缩包。解压后仍需手动完成全部配置与启动步骤。具体步骤参考第二章安装部署。

以下介绍三种格式安装包的解压步骤。

1. tar包解压:

直接解压到当前目录下

```
tar -xzvf tdm_XXX.tar.gz
```

解压到指定目录如/opt 目录下

```
tar -xzvf tdm_XXX.tar.gz -C /opt
```

2. rpm包解压:

不指定目录，默认解压到/opt 目录下

```
rpm -ivh tdm_XXX.rpm
```

指定目录解压如/opt/TDM 目录下

方式一:

```
rpm -ivh --prefix=/path tdm_XXX.rpm
```

方式二:

```
rpm -ivh --relocate /opt/TDM=/path tdm_XXX.rpm
```

使用以下命令可以删除安装包

```
rpm -e tdm_sa
```

3. deb包解压:

不指定目录，默认解压到/opt 目录下

```
dpkg -i tdm_XXX.deb
```

指定目录解压如/opt/TDM 目录下

```
dpkg -x tdm_XXX.deb /opt/TDM
```

使用以下命令可以删除安装包

```
dpkg -r tdm
```

安装更新许可证

用户安装TDM时，可导入相应的许可证（License）。License的载体是一个加密key文件，内容是达梦公司对用户使用TDM软件的授权。安装更新License的操作如下：

1.初次部署TDM，License安装

1) 在解压安装包后进入TDM程序安装目录,假设安装目录为/opt/dmncdb/tdmXXX

```
cd /opt/dmncdb/tdmXXX/bin
```

2) 将tdm_license.key文件拷贝到该目录下，替换默认的tdm_license.key文件，继续配置部署即可。默认的key文件只支持短期的试用。

2.已部署TDM，License更新

1) 找到TDM程序安装目录,假设安装目录：/opt/dmncdb/tdmXXX

```
cd /opt/dmncdb/tdmXXX/bin
```

2) 将tdm_license.key文件拷贝到该目录下，替换默认的tdm_license.key文件；如果是主备集群的环境，则需要替换每个节点该路径下面的key文件。

3) 重启数据库（非必须）

说明：许可证名称为唯一标识请勿修改，可将该目录原有的tdm_license.key文件做好备份，更新license时，请将收到的license文件重命名为tdm_license.key后替换使用。如果保证在更新key时，新key的路径和文件名保持不变。可以无需重启数据库、替换后等待约一分钟后查询是否更新。

3.查看License日期

登录数据库后执行以下命令：登录方式参考启停和登录

```
TDM>show license;
```

安装包目录及配置文件介绍

将TDM的安装包解压。

进入解压后的tdm_XXX目录，其中主要包含bin、config、install、Runtime、script、doc文件夹以及软件产品授权协议（license_zh.txt）。

1. bin文件夹中包含TDM各服务的可执行文件和tdm_license.key(许可证(license))。
2. install文件夹包含安装、更新、卸载脚本。
3. Runtime文件夹包含数据库运行所需的库文件。
4. script文件夹包含服务器启动、停止、登录等被调用的脚本。
5. license_zh.txt: 软件产品授权协议，安装使用TDM前请阅读软件产品授权协议。
6. doc为用户手册和安装手册存放路径。
7. config文件夹中包含所有服务的配置文件，列举说明主要配置文件如下。

base_cfg.ini为数据库服务配置文件，列举主要配置项，可根据实际情况配置，否则使用默认配置。

表2.1 base_cfg.ini主要配置参数说明

参数模块	配置参数	参数说明
[path]	log_dir	数据库日志目录，默认为proc_log,是相对路径,最终路径相对于安装路径 install_dir/TDM_0*。实际使用时可以配置为有权限的绝对路径。
	data_dir	数据库数据目录，默认为Data, 是相对路径，最终路径相对于安装路径 install_dir/TDM_0*。实际使用时可以配置为有权限的绝对路径。
	hot_data_dir	冷热数据分级存储配置,该配置项默认关闭的，打开该配置则开启冷热数据分级存储，是热数据存储路径，默认为HotData，是相对路径最终路径相对于安装路径 install_dir/TDM_0*。实际使用时可以配置为有权限的绝对路径。
	tmp_dir	数据库的临时文件
[base]	timezone	时区配置，默认Asia/Shanghai。
	slow_log_timeout	慢日志，默认值为 0，表示不开启慢日志记录。
[config]	license	许可证（license）的路径配置，默认bin/tdm_license.key,一般不要改动。
	message	数据库函数输出信息id和格式的映射，无需配置改动
	log_properties	日志输出格式，一般无需配置改动
	postgres	兼容pg 的postgres.bki文件路径一般无需改动
	postgres_cfg	兼容pg 的postgresql.conf 路径一般无需改动
	ncdb	ncdb.bki文件路径一般无需改动
[query]	number_threads	查询使用的线程数，默认为16。根据实际环境可以适当增加，可以提升并发性能。
	query_cache_enabled	默认值false、配置true开启数据库的结果集缓存功能
	query_cache_row_limit	默认值1000、配置缓存结果集的最大行数限制
	export_threads	导出数据使用的线程数，默认值为8。
	interactive_query	True，可以开启该参数，来优化结果集的查询。
	dimension_filter	true开启基于dimension信息的索引优化。
[storage]	enable_compression	是否启用压缩（true/false），默认值为true即开启压缩。
	load_data_threads	数据装载线程。

参数模块	配置参数	参数说明
	encrypt_type	默认值plain（非存储加密）、AES_128_CBC（存储加密）。
	ts_retention_check_timeout_sec	默认300s、数据保留策略定时任务的时间
	disk_usage_ratio_limit	[storage]项的配置，数据库默认0不做限制。参数值0~1的小数，磁盘超过这个占比就拒绝写入。
	Disk_free_byte_limit	[storage]项的配置，数据库默认0不做限制。单位是KB，比如设置 $1\ 1024\ 1024 * 1024 = 1073741824$ ，即磁盘空闲少于1G就拒绝写入。
[raft]	thread_pool_size	默认8，可以根据服务器配置修改为16，对分布式的查询写入性能提升有帮助；可以提升load数据速度。Taft模块一般也只可能会修改这个参数。
	log_store_cache_size	log store 缓存 entry的数量，默认值为16。
	heart_beat_interval	Leader向Follower 发送心跳的时间间隔，用于维持 leadership 地位，默认值为500ms。
	election_timeout_lower_bound	选举超时的下限值，Follower 在此时间内未收到心跳会开始选举，默认值为2000。
	election_timeout_upper_bound	选举超时的上限值，实际选举超时会在 [2000, 4000]ms 之间随机选择，避免选举冲突，默认值为4000。
	max_append_size	每次 append_entries 请求中最多包含的日志条目数，控制单个 RPC 的大小，默认值为2。
	max_log_gap_in_stream	流模式下允许的最大日志差距，作为流控机制，默认值为10。
	log_sync_batch_size	新节点加入时，日志同步的批处理大小，默认值为5。
	max_bytes_in_flight_in_stream	流模式下允许的最大飞行字节数，限制网络传输带宽，默认值500MB。
	parallel_log_appending	是否启用并行日志追加，默认值为false、若为 true需要实现异步磁盘写入。
	snapshot_distance	快照创建的日志距离，默认值为2000。每提交 2000 条日志创建一次快照。
	reserved_log_items	日志压缩时保留的最小日志条目数，防止过度压缩，默认值为500。
	use_bg_thread_for_snapshot_io	是否使用后台线程进行快照 I/O 操作，减少主线程阻塞。默认值为true。
	enable_randomized_snapshot_creation	是否启用随机化快照创建，避免集群中多个节点同时创建快照，默认值为true。

参数模块	配置参数	参数说明
	auto_adjust_quorum_for_small_cluster	当集群大小为 2 时，若其中一个节点离线，自动将法定人数调整为 1。默认为true。
	client_req_timeout	客户端请求的超时时间，默认值为3000。
	log_sync_stop_gap	日志同步停止差距，当新节点与 Leader 的日志差距小于此值时，停止使用日志同步，改用 appendEntries，默认值为64。
	fresh_log_gap	判断节点为"新鲜"的日志差距阈值，默认值为32。
	stale_log_gap	判断节点为"过时"的日志差距阈值，默认值为100。
	rpc_failure_backoff	RPC 失败时的退避时间，避免频繁重试导致网络拥塞，默认值为300。
[console]	max_column_width	client里表格式显示的最大列宽，超过这个列宽的部分将用省略号表示。默认值为100。
[sql_server]	number_threads	engine执行命令线程数，根据实际环境可以适当增加，可以提升并发性能。默认值为16。
[rpc_server]	number_threads	rpc message dispatch 线程数量。默认值为256。
	call_log	rpc call 日志开关。默认值为true。
	network_log	rpc network 日志开关。默认值为false。
	call_timeout	rpc call request 超时时间。针对分布式并发多节点load数据的场景，若采集点数量较多（例如存在100万个采集点的数据文件），可通过适当调大超时时间，以支持数据加载操作。默认值为600。
	max_pending_size	一个rpc connect 同时最多发送的消息数量

cluster_cfg.ini 是数据库集群配置文件，列举主要配置项，可根据实际情况配置，否则使用默认配置。

表2.2 cluster_cfg.ini主要配置参数说明

配置参数	参数说明
port	数据库engine服务端口，默认为16501。
min_port max_port	RPC端口使用范围、默认16000-16500。
hosts	数据库各服务安装地址，分布式部署的话duogeip地址用英文逗号分隔开。
number_of_cubics	数据存储分片个数、默认9。按集群规划配置，推荐：集群节点个数 $3N$ ($N \geq 1$)。
number_of_backups	数据副本、每组备份节点个数、默认3。配置1为单副本，集群没有单节点故障能力，副本数多会影响写入性能配置。

install_cfg.ini 是安装配置文件，列举主要配置项，可根据实际情况配置，否则使用默认配置。

表2.3 install_cfg.ini主要配置参数说明

配置参数	参数说明
install_dir	数据库安装路径例如：/opt/TDM。最终安装位置：/opt/TDM/TDM。
username	安装数据库服务节点的用户名。
password	安装数据库服务节点的用户名对应密码。
port	ssh端口默认22。

pg/postgresql.conf 是连接和传输设置的配置文件，列举主要配置项，可根据实际情况配置，否则使用默认配置。

表2.4 postgresql.conf主要配置参数说明

配置参数	参数说明
port	数据库（集群）对外连接端口、默认16502。
max_connection = 100	最大连接数，默认100
ssl =on	通信加密开关、默认注释掉的。
ssl_cert_file='server.crt' ssl_key_file ='server.key'	服务器证书和私钥文件配置，证书和私钥文件放在pg文件夹下和 postgresql.conf 同级。

完成安装后会生成集群信息配置文件config.xml。

8. data文件夹在执行安装后生成，是安装记录，json文件会记录最新一次安装的各文件的MD5值。

单节点模式部署

配置安装

TDM支持远程部署安装（部署目标服务器和安装包所在服务器不同）和本地部署安装（部署目标服务器为安装包所在服务器）。

远程部署配置

以默认端口安装在/opt/TDM为例。

注意：安装包所在服务器需提前安装 paramiko 库 进入安装包目录 tdm_XXX。

1.配置config/pg/postgresql.conf

```
port =16502
```

如需通信加密功能，打开以下三个配置的注释

```
ssl =on

ssl_cert_file='server.crt'

ssl_key_file ='server.key'
```

2.配置config/cluster_cfg.ini

```
port =16501

hosts =远程目标服务器地址
```

3.配置config/install_cfg.ini、

```
install_dir = /opt/TDM （需确保安装目录有权限）

username = 安装用户（远程部署使用该用户）

password = 安装用户的密码

port =22
```

本地部署配置

1.配置config/pg/postgresql.conf

```
port =16502
```

如需通信加密功能，打开以下三个配置的注释

```
ssl =on

ssl_cert_file='server.crt'

ssl_key_file ='server.key'
```

2.配置config/cluster_cfg.ini

```
port =16501

hosts =具体的本地服务器地址    #如果使用127.0.0.1或者0.0.0.0需要确保有paramiko库
```

3.配置config/install_cfg.ini、

```
install_dir = /opt/TDM （需确保安装目录有权限）

username = 安装用户（本地部署用户为当前登录执行脚本的用户，不会获取该配置用户）

password = 安装用户的密码

port =22
```

说明：本地部署用户为当前登录用户，如果配置host为127.0.0.1或者0.0.0.0需要确保有paramiko库

1. 一键部署安装

```
cd install

./install
```

说明：在执行安装脚本后，会进行检查端口是否占用、然后部署安装。脚本无报错执行结束会打印出安装路径、以安装在/opt/TDM>下为例、安装完成后数据库路径为：/opt/TDM/TDM_00。

安装后数据库实例目录介绍

安装部署后进入数据库目录的/opt/TDM/TDM_00，其中主要包含bin、config、Runtime、script文件夹以及start、stop、info、login等脚本。

● bin/

包含TDM各服务的可执行文件和tdm_license.key（许可证(license)）。

● config/

包含所有服务的配置文件，在安装之后主要使用config.xml, 所以如果需要对已经安装好的数据库修改配置，需要通过安装包更新，参考修改配置说明。

● Runtime/

包含服务依赖的全部第三方库。

● Script/

包含各种服务脚本常用如下：

1. start 一键启动脚本。
2. stop 一键停止脚本
3. info 查看数据库服务状态
4. login连接数据库客户端

启停和登录

进入安装目录：/opt/TDM/TDM_00

1. 启动数据库

```
cd /opt/TDM/TDM_00

./start 或 ./tdm start
```

2. 查看数据库服务状态

```
./info 或 ./tdm info
```

3. 登录数据库

使用自带客户端：

```
./login 或 ./tdm login
```

```
./login -U 用户名 -P '密码'      #或者指定用户名密码登录。
```

默认登录用户是postgres，密码为“dameng”，输入密码后即可进入TDM命令程序。在命令行程序中使用SQL命令来创建、删除数据库表以及进行数据的插入查询等。成功进入命令行程序将出现TDM命令行提示符：

```
*****  
  
Welcome to TDM 2.1  
  
Copyright (c) 2024-2025  TDM Inc. All rights reserved.  
  
*****  
  
TDM>
```

或使用psql工具（psql工具的安装附在本小节后）：

```
psql -h 10.14.1.25 -p 25432 -U postgres sslmode=disable
```

4. 停止数据库服务

```
./stop 或./tdm stop
```

psql工具安装：

对于Debian或者Ubutun系统：

1) 使用apt包管理器安装PostgreSQL客户端（psql工具）：

```
sudo apt update  
  
sudo apt install postgresql-client
```

2) 若是离线环境，可以使用随TDM安装包提供的下载好的包安装：

一般放在psql文件夹中，安装psql文件夹中的包*.deb

```
dpkg -i *.deb
```

如果有报错需要修复依赖，执行以下命令：

```
apt-get install -f
```

主要使用RPM包格式的系统例如Centos、RHEL等)

1) 使用yum包管理器的系统安装PostgreSQL客户端（psql工具）：

```
sudo yum install postgresql
```

2) 若是离线环境，可以使用随TDM安装包提供的下载好的包安装：

一般放在psql文件夹中，安装psql文件夹中的包*.rpm例如：

```
yum install postgresql-13.12-1.oe2203.x86_64.rpm
```

或者：

```
rpm -ivh postgresql-13.12-1.oe2203.x86_64.rpm
```

验证安装完成：

```
psql --version
```

如果安装成功，执行上述命令可以看到客户端版本信息。

分布式集群部署

分布式集群概述

分布式时序数据库集群，采用计算和存储分离的分布式架构，分布式架构主要由存储服务器集群、计算服务器集群、管理服务集群以及各种管理组件组成。

支持两种可靠性存储引擎。同构备份引擎通过Raft协议实现高度可靠性，异构备份引擎既满足多副本备份的要求，又为数据库提供多种存储模式服务，可以极大地提升数据库性能。

分布式集群中所有节点对等，每个节点实现微数据库服务，支持SQL查询和数据存储。各节点之间资源相互独立，结合数据本地化存储机制构成高度并发蜂窝计算网络。各种计算引擎（查询引擎、处理引擎等）实现资源调度，将复杂计算转换为网格计算，通过数据路由分发到蜂窝计算网络。通过数据路由，系统实现100%无共享体系结构，便于横向扩展，整个集群无单点故障源。其架构如图2.1所示：

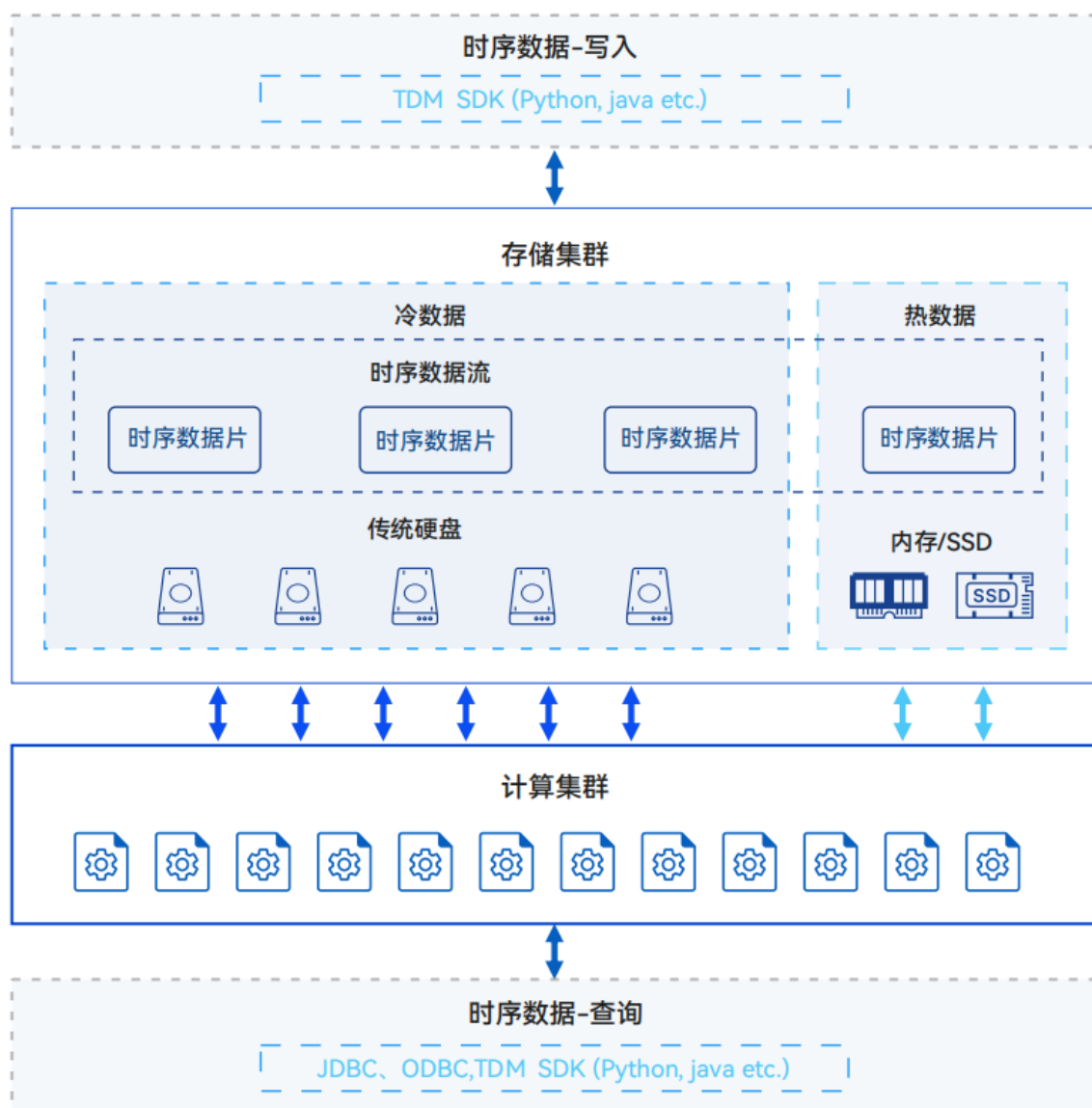


图2.1 分布式集群架构图

分布式搭建

节点规划

搭建3个节点的分布式集群推荐准备3台相同架构和操作系统的服务器，三个节点的engine服务端口一样，我们使用默认的16501；router端口一样，使用默认的16502。

示例集群节点规划如表2.5所示

表2.5节点规划

节点	地址	服务	端口
TDM_00（分布式集群0号节点）	10.14.1.23	admin	-
		store	-
		engine	16501
		router	16502
TDM_01（分布式集群1号节点）	10.14.1.24	admin	-
		store	-
		engine	16501
		router	16502
TDM_02（分布式集群2号节点）	10.14.1.25	admin	-
		store	-
		engine	16501
		router	16502

说明：当前分布式集群部署需要使用相同的用户安装，所以在规划准备节点时，请为集群的节点准备好用户名和密码均相同的安装用户；并且各个节点的安装路径也相同，保证各个节点的安装路径均有权限；为了保证集群所有节点可以执行启停脚本，在准备节点时，确保各个节点都已经安装了Python3.7 及必需的依赖库，详细内容请参考[检查python版本及依赖库](#)

配置安装分布式集群

在安装TDM分布式集群前，需先完成基本的配置，同样以默认端口安装在/opt/dmncdb为例。我们将安装包放在集群中的一台服务器上例如：10.14.1.23.

按照规划的集群信息配置集群信息，完成基础的配置，详细的配置说明请参考[安装包目录及配置文件介绍](#)

1. 进入安装包tdm_XXX配置集群各端口、节点地址、安装目录、安装用户等

配置config/pg/postgresql.conf

```
port =16502
```

如需通信加密功能，打开以下三个配置的注释

```
ssl =on

ssl_cert_file='server.crt'

ssl_key_file ='server.key'
```

配置config/cluster_cfg.ini 、通过配置hosts确定搭建分布式集群的节点ip

```
port =16501

hosts =10.14.1.23,10.14.1.24,10.14.1.25

[storage_cluster]

number_of_cubics = 9（按集群规划配置，一般为集群节点个数*3*N（N>=1））

number_of_backups = 3（集群副本数，默认3，1为单副本）
```

配置config/install_cfg.ini、

```
install_dir = /opt/dmncdb（需确保安装目录有权限）

username = test

password = test23

port =22
```

说明：当前分布式集群部署需要使用密码相同的用户安装，并且各个节点的安装路径也相同，保证各个节点的安装路径均有权限。

一键部署安装

```
cd install

./install
```

说明：在执行安装脚本后，会进行检查端口是否占用、然后部署安装。脚本无报错执行结束会打印出安装路径、以安装在/opt/dmncdb下为例、安装完成后登录到各个节点数据库路径为：分布式集群0号节点：/opt/dmncdb/TDM_00；分布式集群1号节点：/opt/dmncdb/TDM_01；分布式集群2号节点：/opt/dmncdb/TDM_02。

不同架构搭建分布式

推荐使用相同架构和操作系统的服务器来搭建分布式，如果需要使用不同架构和操作系统的服务搭建，需要准备各个对应系统架构的安装包。

按照节点规划准备好服务器，确保各个节点都已经安装了Python3.7 及必需的依赖库，详细参考[检查python版本及依赖库](#)。

配置安装集群参考配置安装分布式集群。在执行install 完成一键部署后，替换和安装包不同架构系统的bin和Runtime文件夹（替换为对应架构和操作系统的，需要注意：替换bin文件夹的时候同时替换好tdm_license.key），替换完成后可以启动集群。

启停和登录

分布式集群的启停、登录、查看集群服务状态信息在任何节点均可以。在0号节点操作为例：进入安装目录：/opt/dmncdb/TDM_00

1. 启动

```
./start 或 ./tdm start
```


2. 查看数据库服务状态

```
./info 或 ./tdm info
```

3. 登录数据库

```
./login 或 ./tdm login
```

```
./login -U 用户名 -P '密码' #或者指定用户名密码登录。
```

默认登录用户是postgres，密码为“dameng”，输入密码后即可进入TDM命令程序。在命令程序中使用SQL命令来创建、删除数据库表以及进行数据的插入查询等。成功进入命令程序将出现TDM命令行提示符：

```
*****

Welcome to TDM 2.1

Copyright (c) 2024-2026 TDM Inc. All rights reserved.

*****

TDM>
```

说明：在任何节点执行登录客户端，都是登录连接整个分布式集群。

4. 停止数据库

```
./stop 或 ./tdm stop
```

集群扩缩容

目前TDM分布式集群扩缩容功能是基于单副本的、即部署集群初始配置 `number_of_backups = 1` ([storage_cluster]的`number_of_backups`) 的集群才支持扩缩容功能，目前扩容支持在线只读扩容，并且扩容节点和集群为相同架构，缩容是离线的，需要停止数据库服务。

集群扩容

对已经部署的TDM的3节点集群（单副本）扩容，增加节点，步骤如下：

进入数据库安装包目录`tdm_XXX/install`，执行以下扩容命令：

```
./addserver --ip **10.14.1.26** --cubic_size 12
```

说明：10.14.1.26是新增节点ip，准备节点时，确保各个节点都已经安装了Python3.7 及必需的依赖库，详细参考[检查python版本及依赖库](#)。`cubic_size`是扩容集群新增cube数（`umber_of_cubics`）。

在执行扩容命令后，会进行检查新节点、更新集群配置、然后部署安装，并且启动服务。以安装在`/opt/dmncdb`下为例、新增的节点为分布式集群3号节点：`/opt/dmncdb/TDM_03`。扩容不迁移历史数据。

集群缩容

对已经部署的TDM的4节点集群（单副本）缩容，减少节点。步骤如下：

1. 数据落盘（进入数据库安装目录，如：/opt/dmncdb/TDM_00）

进入客户端

```
./login
```

执行落盘命令：

```
flush all
```

2. 停止集群（进入数据库安装目录，如：/opt/dmncdb/TDM_00）

```
./stop 或./tdm stop
```

3. 进入数据库安装包目录tdm_XXX/install，执行以下缩容命令

```
./addserver --ip 10.14.1.25
```

说明：在执行扩容命令后，会自动迁移数据，更新集群配置。缩容成功完成可删除缩容节点上的安装目录。

3. 启动集群。

启停，查看状态参考启停和登录。

部署后实例目录说明

完成安装后进去数据库安装目录，path/TDM_0*(单机点为TDM_00， 分布式根据节点数分别是TDM_00、TDM_01、TDM_02.....)。

其中主要包含bin、config、Runtime、script、Data、proc_log、proc_tmp、和clean/console/info/login/show/start/stop/tdm这些执行命令文件

1. bin文件夹中包含TDM各服务的可执行文件和tdm_license.key(许可证(license))。
2. Runtime文件夹包含数据库运行所需的库文件。
3. script文件夹包含服务器启动、停止、登录等命令调用的脚本。
4. clean/console/info/login/show/start/stop等是具体的命令，tdm是集成了所有命令，执行命令时调用script中脚本。
5. Data数据文件夹（未配置具体绝对路径，使用默认配置）。
6. proc_logs数据库日志文件夹（未配置具体绝对路径，使用默认配置）。
7. proc_tmp临时文件。

TDM版本查看

想要确认已经安装部署的TDM 的具体版本信息，登录数据库执行命令：

```
TDM>select db_version();
```

说明：输出信息可以显示TDM 版本号，打包系统、打包日期、代码版本。使用 select version(); 命令查看的是兼容PG的信息。

更新或修改数据库配置

已经安装启动的数据库需要修改配置、或者数据库更新（二进制、脚本等）需要执行以下操作。

1. 停止数据库服务

```
cd /opt/TDM/TDM_00    #进入安装目录

./stop 或 ./tdm stop
```

2. 修改配置或者替换更新文件

进入安装包tdm_XXX/config目录、修改目标配置，或者替换更新文件（key或者二进制文件）。

说明：对于已经有数据的数据库集群，请不要变更集群信息相关的配置（cluster_cfg.ini）否则会造成集群前后不一致。

3. 将变更的配置或者新文件更新到安装的实例

```
cd tdm_XXX/install

./install -o -f    #生成新的config.xml文件
```

4.更新全部配置文件或者新文件到数据库安装目录

```
./update
```

5. 启动数据库

```
cd /opt/TDM/TDM_00 #进入安装目录

./start 或 ./tdm start
```

卸载

在卸载TDM前，需先确认安装包的配置和安装信息一致（安装服务器地址：hosts，安装路径：install_dir，如果是远程部署需要确认安装用户和密码信息）。

1. 停止数据库（集群）

```
cd /opt/TDM/TDM_00

./stop 或 ./tdm stop
```

2. 清理数据（如果需要）

```
cd install

./clean
```

等待执行完成，会删除Data、proc_log、proc_tmp。

3. 卸载（进入安装包目录tdm_XXX）

```
cd install  
  
./uninstall
```

等待执行完成，即可完成对数据库服务的卸载，数据文件Data和日志文件。默认不会在卸载时删除。

SQL语言简介

结构化查询语言 SQL (Structured Query Language) 是在 1974 年提出的一种关系数据库语言。由于 SQL 语言接近英语的语句结构，方便简洁、使用灵活、功能强大，备受用户及计算机工业界的欢迎，被众多计算机公司和数据库厂商所采用，经各公司的不断修改、扩充和完善，SQL 语言最终发展成为关系数据库的标准语言。

TDM支持大部分 SQL 标准，集数据定义、数据查询、数据操纵和数据控制于一体，是一种统一的、综合的关系数据库语言。它功能强大，使用简单方便、容易为用户掌握。本文主要介绍 TDM 系统所支持的 SQL 语言-TDM_SQL 语言。

标识符和关键字

标识符用于标识表、列或者其他数据库对象。标识符的长度目前不受限制。关键字即SQL语言中具有特定意义的词，例如SELECT，WHERE等，关于关键字的完整列表参见附录1。

标识符包含两大类，一种是不被双引号修饰的普通标识符和一种被双引号修饰的受限标识符。普通标识符必须以字母开始，后续字符可以是字母、下划线（`_`）、数字。例如`A`，`test1`，`TABLE_B`。受限标识符可以是由双引号包围的任意字符序列。

关键字和不被引号修饰的标识符是大小写不敏感的。且不被引号修饰的标识符总是转换成小写，例如标识符`FOO`、`foo`和“`foo`”被认为是相同的。例如：

```
UPDATE MY_TABLE SET A=5;
```

可以等价地写成：

```
UpDaTE my_TABLE SeT A=5;
```

常见的习惯是将关键词写成大写，而名称写成小写，例如：

```
UPDATE my_table SET a=5;
```

常量

在TDM_SQL中有两种隐式类型常量：字符串和数字。常量也可以显示指定类型。

字符串常量

字符串常量是由单引号包围的任意字符序列，例如`'This is a string'`。如果要在一个字符串中包含一个单引号，可以写两个相连的单引号，例如`'Dianne"s horse'`。

数字常量

数字常量的表示形式有以下几种情况：

```
_digits_  
  
_digits_[_digits_][e[+_]_digits_]   
  
[_digits_]_digits_[e[+_]_digits_]   
  
_digits_[e[+_]_digits_]
```

其中digits是一个或多个十进制数字（0 到 9）。如果使用了小数点，在小数点前面或后面必须至少有一个数字。如果含有指数标记（e），在其后必须跟着至少一个数字。

数字常量的例子：42、3.5、4.、.001、5e2、1.925e-3。

数字常量具有的数据类型视其值大小而定。基本类型是i64，若值超过i64的最大值但在u64值域内，则其类型为u64；否则其类型为decimal。

表达式

在TDM中值表达式可以应用到各种场景中，例如应用在SELECT命令的目标列表中、作为INSERT中的新列值或者若干命令中的搜索条件。值表达式也被称为标量表达式（或者甚至简称为表达式）。表达式中允许使用算术运算符、逻辑运算符等。

1. 值表达式可以是一个常量，例如'this is a string'；
2. 值表达式可以是对列的引用，形式如下：

```
tablename.columnname
```

tablename是一个表的名字，或者是在FROM子句中为一个表定义的别名。

1. 值表达式可以是一次对操作符的调用，例如a+b；
2. 值表达式可以是一次对函数的调用，例如now()；
3. 值表达式可以是一个聚集表达式，例如count(*)；
4. 值表达式还可以是一个刚好返回一行一列的标量子查询（关于标量子查询参见6.8小节），使用时需要使用括号。

数据类型

目前TDM支持的数据类型包括多种数值类型、字符串类型、布尔类型，以及时间类型。

数值类型

数值类型包含不同长度的整数类型以及可选精度的小数。表3.1列出了支持的全部数值类型。

表3.1 数值类型

名字	别名	说明	存储宽度
I8	TINYINT	8位整数，取值范围[-128, 127]	1字节
U8	TINYINT UNSIGNED	8位无符号整数，取值范围[0, 255]	1字节
I16	SMALLINT	16位整数，取值范围[-32768, 32767]	2字节
U16	SMALLINT UNSIGNED	16位无符号整数，取值范围[0, 65535]	2字节
I32	INT, INTEGER	32位整数，取值范围[-2147483648, 2147483647]	4字节
U32	INT UNSIGNED, INTEGER UNSIGNED	32位无符号整数，取值范围[0, 4294967295]	4字节
I64	BIGINT	64位整数，取值范围[-2 ⁶³ , 2 ⁶³ -1]	8字节
U64	BIGINT UNSIGNED	64位无符号整数，取值范围[0, 2 ⁶⁴ -1]	8字节
DECIMAL	NUMERIC, NUMBER	取值范围[-2 ¹²⁷ , 2 ¹²⁷ - 1]，最大有效数字为38位，可精确表示小数	17字节
FLOAT	FLOAT	可变精度，不精确	4字节
DOUBLE	DOUBLE	可变精度，不精确	8字节

字符类型

TDM目前支持的字符类型如表3.2所示。

表3.2 字符串数据类型

名字	别名	说明	存储宽度
STRING	TEXT	字符串类型	变长

目前TDM服务端存储仅支持utf-8字符集，客户端会通过TDM-ROUTER转码，支持常用的字符集。

日期时间类型

TDM目前支持的日期时间类型如表3.3所示。

表3.3 时间数据类型

名字	别名	说明	存储宽度
DATE	无	只包含日期部分（年-月-日）	4字节
TIME	无	只包含时间部分（时:分:秒），或时间间隔	8字节
DATETIME	无	包含日期和时间组合（年-月-日 时:分:秒）	8字节
TIMESTAMP	无	包含日期和时间组合（年-月-日 时:分:秒）	8字节

目前时间类型的输入格式支持'YYYY-MM-DD HH:MM:SS:SI '和'YYYY-MM-DDTHH:MM:SS:SI '，且使用时需要用单引号包围，例如'2024-01-02 00:00:00'。

TDM目前不支持interval 但是支持将微秒级整数和时间戳之间相互转换结合数学运算实现。微秒级整数转为时间戳例如：1742353888911000::timestamp。

一个具体的例子：查询当前时间的前100分钟的数据，需要将时间转换为微秒级整数进行数学运算后再转回时间戳：

```
TDM>select cola,colb,ts from table1 where ts >=(now()::i64-100*60*1000000)::timestamp
limit 100;
```

布尔类型

TDM提供标准的SQL类型布尔类型，如表3.4所示。

表3.4 布尔类型

名字	别名	说明	存储宽度
BOOL	BOOLEAN	状态为真或假。	1字节

布尔类型可以有多个状态：true（真）、false（假）和第三种状态unknown（未知），未知状态由SQL空值表示。目前，语法中接受“true”表示真，“false”表示假。

数据定义

本章介绍 TDM 的数据定义语句，包括数据库的创建及查看语句、表创建、表修改、表查看，表删除以及模式定义等。

管理数据库

创建数据库

通过以下语句创建数据库。

语法格式

```
CREATE DATABASE [IF NOT EXISTS] db_name
```

使用说明

1. 输入的数据库名等标识符名默认不区分大小写，并且在数据库内会自动以小写形式存储。
2. 如果未使用 IF NOT EXISTS 关键字，则用该语句创建已存在的数据库时会返回错误。

举例说明

例1：创建名为test的数据库。

```
TDM>CREATE DATABASE test;  
  
Successfully create database
```

使用数据库

通过USE语句指定后续语句使用的默认数据库。

语法格式

```
USE database
```

举例说明

例1：创建名为test的数据库，并切换到该数据库。

```
TDM>CREATE DATABASE test;  
  
Successfully create database  
  
TDM>USE test;  
  
Database changed.
```

查看数据库

通过该语句查看当前存在的所有数据库。该语句会以表格形式返回当前存在的所有数据库的名称与创建时间。

语法格式

```
SHOW DATABASES
```

举例说明

例1：创建名为test的数据库，并查看数据库。

```
TDM>CREATE DATABASE test;
```

Successfully create database

```
TDM>SHOW DATABASES;
```

Database	Timestamp
DB	2023-08-18 07:00:42.811257
TEST	2023-08-22 08:02:11.870905

管理表

数据库中的表由行和列组成。列的数量和顺序是固定的，并且每一列拥有一个名字。行的数目是变化的，它反映了在一个给定时刻表中存储的数据量。TDM数据库表分为普通表和时序表两类。普通表即传统的关系型数据库表，时序表是具有特殊功能的表，可以更高效地处理时序类数据，时序表的使用和普通表几乎一致。

表定义语句

定义普通表

语法格式

```
CREATE TABLE [IF NOT EXISTS] table_name([column_name data_type [NULL][DEFAULT  
default_expr] } [,...]))
```

参数

1. table_name：要被创建的表名。
2. column_name：新建的列的名称。
3. data_type：列的数据类型，有关TDM支持的数据类型参考3.4小节。
4. NULL：使用该参数表示列允许插入空值，该参数默认可以省略，表示默认情况不允许包含空值。
5. DEFAULT：该子句用来为列赋予一个默认数据，其数据类型需要与列类型保持一致，当插入或更新某一行并且没有指定值时，将会被设定为默认值。如果建表时没有指定默认值，且指定列不允许插入空值，则数值类型默认值默认为0，字符串类型默认值为空字符，布尔类型默认值为false，时间戳类型默认值为“1970-01-01 00:00:00”；如果指定列允许插入为空，则在没有指定默认值时，默认值为NULL。

使用说明

1. 表和列的名字遵循3.1小节中解释的标识符语法。表的数据列需要指定列名、列的数据类型；
2. 默认所有数据列不允许写入空值；向非空列写入空值时会报错。

举例说明

例1：创建users表，所有列默认为非空。

```
TDM>CREATE TABLE users(name string,age int);
```

Successfully create table

例2: 创建USERS表, 所有列默认为非空。

```
TDM>CREATE TABLE "USERS"(name string,age int);
```

Successfully create table

例3: 创建department表, 部门名称列可以为空。

```
TDM>CREATE TABLE department(depID int,depName string null);
```

Successfully create table

定义时序表

语法格式

```
CREATE HYPERTABLE table_name ([{col_name col_type [NULL][DEFAULT default_expr] AS  
{DIMENSION | MEASURE} `time_col` TIMESTAMP [DEFAULT default_expr]} [, ...]])  
[PARTITION_INTERVAL N 单位]
```

参数

1. table_name: 要被创建的表名。
2. col_name: 新建的列的名称。
3. col_type: 列的数据类型, 有关TDM支持的数据类型参考3.4小节。
4. NULL: 该列允许插入空值, 该参数默认可以省略, 表示默认情况不允许包含空值。
5. DEFAULT: 该子句用来为列赋予一个默认数据, 其数据类型需要与列类型保持一致, 当插入或更新某一列并且没有指定值时, 将会被设定为默认值。如果建表时没有指定默认值, 且指定列不允许插入空值, 则数值类型默认值默认为0, 字符串类型默认值为空字符, 布尔类型默认值为false, 时间戳类型默认值为“1970-01-01 00:00:00”; 如果指定列允许插入为空, 则在没有指定默认值时, 默认值为NULL。
6. AS DIMENSION: 指定该列存储时序数据的元信息属性值;
7. AS MEASURE: 指定该列存储时序数据的指标测量值;
8. time_col: 指定该列存储时序数据产生的时间。
9. PARTITION_INTERVAL N 单位: 该部分参数指定时间分区, 将存储间隔修改为N天、N月、N年。N只支持正整数, 单位仅支持day /month/ year。不指定该参数时默认分区为1年。

使用说明

1. 表和列的名字遵循3.1小节中解释的标识符语法。表的数据列需要指定列名、列的数据类型;
2. 数据列必须使用 AS DIMENSION 或 AS MEASURE 来指定非时间戳列的属性;
3. 默认所有数据列不允许写入空值, 向非空列写入空值时会报错;
4. 至少包含有一个 MEASURE列;
5. 必须且仅包含一列时间戳类型数据列 (time_col) 。
6. time_col: 不允许为空。

举例说明

例1: 创建关于天气的weather表, 所有列默认为非空。

```
TDM>CREATE HYPERTABLE weather(location string as dimension,temperature integer as
measure,`time` timestamp);
```

Successfully create table

修改表

TDM提供了一组命令来对已有的表的结构进行修改。包括增加列，移除列，修改默认值，修改列数据类型，重命名列，重命名表。目前只支持对时序表结构的修改。

增加列

通过该语句可以为表新增一个列。

语法格式

```
ALTER TABLE table_name ADD [COLUMN]column_name data_type [NULL][DEFAULT default_expr]
[AS MEASURE];
```

使用说明

1. 新增的列将被默认值所填充。
2. 关键字COLUMN可以省略。
3. 为时序表新增列必须带AS MEASURE，且时序表只能新增MEASURE列。

举例说明

例1：为表weather新增一列。

```
TDM>ALTER TABLE weather ADD COLUMN newcol int as measure;
```

Successfully alter table

移除列

通过该语句可以删除表中的一个列。

语法格式

```
ALTER TABLE table_name DROP [COLUMN] column_name;
```

使用说明

1. 删除该列，且列中数据将会消失，涉及该列的表约束也会被移除。
2. 关键字COLUMN可以省略。

举例说明

例1：删除表weather中的一列。

```
TDM>ALTER TABLE weather drop column newcol;
```

Successfully alter table

更改列的默认值

通过该语句可以为列设置一个新的默认值。

语法格式

```
ALTER TABLE table_name ALTER [COLUMN] column_name SET DEFAULT default_expr;
```

使用说明

1. 以上操作不会影响任何表中已经存在的行，只是改变了后续插入或更新的默认值。
2. 关键字COLUMN可以省略。

举例说明

例1：将表weather中的newcol列的默认值设置为1。

```
TDM>ALTER TABLE weather alter newcol set default 1;
```

```
Successfully alter table
```

删除列的默认值

通过该语句移除列的默认值。

语法格式

```
ALTER TABLE table_name ALTER [COLUMN] column_name DROP DEFAULT
```

使用说明

1. 以上操作不会影响任何表中已经存在的行，只是改变了后续插入或更新的默认值。
2. 以上操作等同于将默认值设置为NULL。
3. 关键字COLUMN可以省略。
4. 时序表不可删除 DIMENSION列和timestamp列的默认值。

举例说明

例1：删除表weather中的newcol列的默认值。

```
TDM>ALTER TABLE weather alter column newcol drop default;
```

```
Successfully alter table
```

修改列的数据类型

通过该语句可以将列转换为其他数据类型。

语法格式

```
ALTER TABLE table_name ALTER [COLUMN] column_name TYPE data_type
```

使用说明

1. 只有当列中的所有记录都能通过隐式转换为新的数据类型时该操作才能成功。
2. 关键字COLUMN可以省略。
3. 时序表不可修改 DIMENSION列和timestamp列的数据类型。

举例说明

例1：将表weather的newcol列转换为字符串类型。

```
TDM>ALTER TABLE weather alter column newcol type string;

Successfully alter table
```

重命名列

通过该语句可以将列重命名。

语法格式

```
ALTER TABLE table_name RENAME [COLUMN] column_name TO new_column_type
```

使用说明

1. 关键字COLUMN可以省略。
2. 时序表不可修改 DIMENSION列和timestamp列的列名。

举例说明

例1：将表中weather中的newcol列重命名为newcolname。

```
TDM>ALTER TABLE weather rename newcol to newcolname;

Successfully alter table
```

重命名表

通过该语句可以将表重命名。

语法格式

```
ALTER TABLE table_name RENAME TO new_name
```

举例说明

例1：将表weather重命名为newweather。

```
TDM>ALTER TABLE weather rename to newweather;

Successfully alter table
```

查看表

通过该语句查看当前存在的所有数据库表。该语句会以表格形式返回当前存在的所有数据库的名称与创建时间。

语法格式

```
SHOW TABLES
```

使用说明

1. 表和列的名字遵循3.1小节中解释的标识符语法。表的数据列需要指定列名、列的数据类型；

举例说明

例1：创建名为weather的数据库表后，查看数据库中存在的表。

```
TDM>CREATE TABLE weather(location string, temperature integer,`time` timestamp);
Successfully create table

TDM>SHOW TABLES;
+-----+-----+-----+-----+
| Schema | Table   | Mode      | Timestamp                               |
+-----+-----+-----+-----+
| public | weather | Relational| 2024-03-26 06:54:56.745699             |
+-----+-----+-----+-----+
```

查看表结构

通过该语句查看指定表的结构。该语句会以表格形式返回表中所有的字段，字段类型，是否可以为空以及默认值等信息。

语法格式

```
DESC table_name
```

举例说明

例1：创建名为test的数据库，并查看数据库。

```
TDM>CREATE HYPERTABLE weather(loc string as dimension,t integer as measure,`ts`
timestamp);

Successfully create table

TDM>DESC weather;
+-----+-----+-----+-----+-----+-----+
| Field  | Mode      | Data Type | Nullable | Default                | Compression |
+-----+-----+-----+-----+-----+-----+
| loc    | dimension | String    | FALSE    |                        | lz4         |
+-----+-----+-----+-----+-----+-----+
| t      | measure   | i32       | FALSE    | 0                      | lz4         |
+-----+-----+-----+-----+-----+-----+
| ts     | timestamp | Timestamp | FALSE    | 1970-01-01 00:00:00    | lz4         |
+-----+-----+-----+-----+-----+-----+
```

删除表

通过该语句可以将数据库中的表删除，同时删除表中数据。

语法格式

```
DROP TABLE table_name
```

举例说明

例1：删除表weather。

```
TDM>DROP TABLE weather;
```

Successfully drop table

模式

TDM支持在一个数据库中创建不同的模式，模式又包含不同的表。不同的模式中可以包含相同名称的表。

语法格式

```
CREATE SCHEMA schema_name
```

使用说明

1. 一个数据库中的模式名不能重名；
2. 每个数据库都有一个默认的模式public，当引用表不带模式名时，默认操作的是public模型下的表。

举例说明

例1：创建模式myschema。

```
TDM>CREATE SCHEMA myschema;
```

Successfully create schema

例2：在myschema模式下创建表。

```
TDM>CREATE TABLE myschema.mytable(col int);
```

Successfully create table

数据操纵

导入数据

TDM支持通过LOAD DATA语句从文件中批量导入数据。

语法格式

```
LOAD DATA {INFILE|INDIR} path INTO TABLE table_name

[FIELDS TERMINATED BY 'string ']

[LINES TERMINATED BY 'string ']
```

使用说明

1. 导入的数据文件：
 1. 必须是csv格式；
 2. 首行必须指定文件中各数据列对应目标表中的列名（列名可以不被引号包裹），并用逗号隔开；
 3. 后续行均为数据行，其中字符串类型数据是否使用双引号包裹均支持；字段值之间使用分隔符，默认以逗号分隔；当字符串内容含包裹符本身时需要使用转义符"转义，字符串内容包含分隔符时不需转义；
 4. 除timestamp列其他的字段值可以为null。
 5. timestamp列数据只接受2种格式目前：不被任何符号包裹的微秒级整数、被包裹符号
2. 通过INFILE path 或者INDIR path指定加载数据的文件名或目录，如果指定目录，则会尝试加载其中所有的文件。在指定数据文件或目录时，支持绝对路径也支持相关路径，相对路径基于TDM安装路径。
3. FIELDS TERMINATED BY 'string' 指定字段值的分隔符，不指定时默认是逗号','。
4. LINES TERMINATED BY 'string' 指定数据行的分隔符，不指定时默认分隔符为'\n'。
5. 在执行load命令后逐条处理数据，会将所有正确的数据load入库，通过client执行load命令后返回信息会统计失败的数据条目，通过查看TDM的store.error查看具体load失败的数据。

举例说明

例1：有数据文件weather.csv，内容如下：

```
"location","temperature","ts"

"suzhou",32,1577836800000000

"shanghai",33,1577836800000000

"kunshan",34,1577836800000000

"hangzhou",32,1577836800000000
```

向前文中创建的weather表中导入路径为/opt/weather.csv的数据文件，命令的执行结果如下所示：

```
TDM>LOAD DATA INFILE '/opt/weather.csv' INTO TABLE weather;
```

```
COPY 4 rows of 1 files using 101.38ms, get 0 errors
```

例2：有换行符为\r\n的数据文件weather1.csv，内容如下：

```
location,temperature,ts

"suzhou";32;"2024-09-01 01:02:03"

"shanghai";33;"2024-09-02 01:02:03"

"kuns\han";34;"2024-09-03 01:02:03"

"hangzhou";32;"2024-09-04 01:02:03"
```

向前文中创建的weather表中导入路径为/opt/weather1.csv的数据文件，命令的执行结果如下所示：

```
TDM>LOAD DATA INFILE '/opt/weather1.csv' INTO TABLE weather FIELDS TERMINATED BY
';'lines terminated by '\r\n';
```

```
COPY 4 rows of 1 files using 101.38ms, get 0 errors
```

数据插入

数据插入语句用于向已定义好的表中插入单个或成批的数据。TDM支持通过INSERT命令向表中写入数据。

语法格式

```
INSERT INTO table_name [(column_name [,...]) VALUES ({expression | DEFAULT}[,...] )
[,...];
```

参数

1. table_name：表名。
2. column_name：列名。
3. expression：要赋予相应列的表达式或值，DEFAULT则相应的列填充默认值。

使用说明

1. 插入数据时，目标列的名称可以以任意顺序列出。如果没有给出列名，则有两种策略确定目标列。第一种是以建表时被声明的顺序列出该表的所有列。另一种是，如果VALUES子句只提供了N个列，则以建表时被声明的顺序列出前N列。
2. 每一个没有出现在插入列表中的列将被默认值填充。如果没有默认值，将被空值填充。
3. 如果数据行中的数据类型与目标表数据列类型不一致，则会尝试进隐式转换，具体包括数值类型与字符串类型之间的互相转换，以及字符串类型转换到时间戳和bool类型，TDM支持将微秒级整数通过隐式转换为时间戳，例如：1742353888911000::timestamp。

举例说明

例1：创建名为weather的数据库表，并向表中插入一条数据。

```
TDM>CREATE HYPERTABLE weather (location string as dimension DEFAULT 'suzhou',
temperature integer as measure,`time` timestamp);
```

Successfully create table

```
TDM>INSERT INTO weather VALUES('suzhou',24,'2024-03-22 00:00:00');
```

```
INSERT 0 1
```

例2：向weather表插入数据，显示的列出列名并向其插入数据，被省略的列将被默认值填充。

```
TDM>INSERT INTO weather(location,`time`) VALUES ('suzhou','2024-03-22 00:00:00');
```

```
INSERT 0 1
```

例3：向weather表插入数据，显示的指定某些列用默认值填充。

```
TDM>INSERT INTO weather VALUES(DEFAULT,25,'2024-03-22 00:00:00');
```

```
INSERT 0 1
```

例4：TDM支持将微秒级整数通过隐式转换为时间戳插入数据。

```
TDM>INSERT INTO weather VALUES(DEFAULT,25,1742353888911000::timestamp);
```

```
INSERT 0 1
```

LineProtocol协议插入数据

Line Protocol 是一种用于插入时序数据的协议。本实现与Influx DB的Line Protocol存在一些区别，TDM插入不支持schemaless。

语法格式

```
table_name , [dimension_column_name=value [,...]] measure_column_name=value [,...]
timestamp
```

参数

1. table_name: 表名。
2. dimension_column_name: dimension列名。
3. measure_column_name:measure列名
4. timestamp: 纳秒为单位的时间戳

使用说明

1. 默认表名、列名、dimension值均不加引号，measure值若有字符串则需要用引号包裹，数据库中表名是大写引用时需要使用双引号包裹。
2. dimension列表和measure列表为“列名=值”的键值对形式，列表的键值对间用逗号分隔。
3. 表名与dimension列表用逗号分隔，若无dimension列，表名与measure列表间用逗号分隔。
4. dimension列表与measure列表间用空格分隔。
5. measure列表与timestamp间用空格分隔。
6. timestamp表现形式：纳秒为单位的以epoch time（1970-01-01）为零点的整数。

7. value中存在空格或逗号等，使用`\\`进行转义；使用`\\`表示单个反斜杠。
8. 每条数据中必须指定所有字段，不支持根据默认值补全。
9. 若measure列的值为null，则在=后面直接留空；dimension不支持null值。
10. 构建Line Protocol数据后，直接结尾加分号作为SQL语句发送即可。
11. 要一次性插入若干条 Line Protocol 数据，则一行写一条 Line Protocol 数据，用`\\n`结尾，最后一行末尾加分号即可。
12. 大量的数据导入使用source命令，将数据按照11点的要求 写入xxx.sql文件，在TDM client使用命令source '/path/XXX.sql' 插入。

举例说明

例1：创建名为weather的数据库表，并向表中插入Line Protocol数据。

```
TDM>CREATE HYPERTABLE weather (location string as dimension DEFAULT 'suzhou',
temperature integer as measure NULL,`time` timestamp);

Successfully create table

TDM>weather,location=suzhou temperature=32 1737010657000000000;

INSERT 0 1

TDM>weather,location=suzhou temperature= 1737010657000000000;

INSERT 0 1

TDM>weather,location="su\\,zhou" temperature= 1737010657000000000;

INSERT 0 1

TDM>weather,location="su\\ zhou" temperature= 1737010657000000000;

INSERT 0 1
```

例2：向weather表中插入多条Line Protocol数据。

```
TDM>weather,location=suzhou temperature=32 1737010657000000000

weather,location=suzhou temperature=32 1737010659000000000

weather,location=suzhou temperature=32 1737010660000000000

weather,location=suzhou temperature=32 1737010661000000000;

INSERT 0 4
```

例3：向weather表使用source命令插入多条Line Protocol数据的文件，将例2中的4条数据放入linep.sql文件中。

```
TDM>source '/testload/linep.sql';

INSERT 0 4
```

数据更新

数据更新语句用于修改数据库中已存在的数据。可以更新单个行，更新一部分行，也可以更新全部行，可以针对一列进行更新，也可以同时更新多个列。

语法格式

```
UPDATE table_name SET {column_name = {expression | DEFAULT}}|  
  
(column_name [,...])=({expression | DEFAULT}{[,...])} [,...]  
  
[WHERE condition]
```

参数

1. table_name: 表名。
2. column_name: 列名。
3. expression: 要赋予相应列的表达式或值，DEFAULT则相应的列填充默认值。
4. condition: 一个返回boolean类型值的表达式。让这个表达式返回true 的行将会被更新。

使用说明

1. UPDATE更新满足条件的所有行中指定列的值。如果不指定WHERE条件，则会有更新所有行。如果给定了WHERE条件，那么只有匹配它后面的条件的行被更新。
2. 通过在SET语句中列出多个赋值表达式，可以同时更新多列，参见例2。
3. 时序表不可更新DIMENSION列和timestamp列的数据。

举例说明

创建表Weather、插入一条数据。

```
TDM>CREATE HYPERTABLE weather(location string as dimension DEFAULT 'suzhou', temperature  
integer as measure,`time` timestamp);  
  
Successfully create table  
  
TDM>INSERT INTO weather VALUES('suzhou',24,'2024-03-22 00:00:00'),('shanghai',23,'2024-  
03-22 00:00:00');  
  
INSERT 0 2
```

例1：以下语句将更新location 字段值为'shanghai'的行的temperature 字段。

```
TDM>UPDATE weather SET temperature = 23.5 where location ='shanghai';  
  
UPDATE 1 rows cost 30.19ms
```

数据删除

数据删除语句用于删除表中的数据，可以删除单个行，删除一部分行，也可以删除全部行。

语法格式

```
DELETE FROM table_name [WHERE condition]
```

使用说明

1. 如果指定了WHERE子句，则只会删除满足WHERE条件的行；如果不指定WHERE子句，则删除全部数据。

举例说明

创建表Weather、插入两条数据。

```
TDM>CREATE HYPERTABLE weather(location string as dimension DEFAULT 'suzhou', temperature integer as measure,`time` timestamp);
```

Successfully create table

```
TDM>INSERT INTO weather VALUES('suzhou',24,'2024-03-22 00:00:00'),('shanghai',23,'2024-03-22 00:00:00');
```

INSERT 0 2

例1：以下语句将删除location 字段值为'shanghai'的行。

```
TDM>DELETE FROM weather where location ='shanghai';
```

DELETE 1 rows cost 66.47ms

例2：以下语句将删除表中全部数据。

```
TDM>DELETE FROM weather;
```

DELETE 1 rows cost 29.38ms

除了使用DELETE命令删除数据外，还可以使用TRUNCATE命令进行数据删除。该命令将删除表中全部数据。

语法格式

```
TRUNCATE [TABLE] table_name
```

例1：以下语句将删除weather表中的全部数据。

```
TDM>TRUNCATE TABLE weather;
```

TRUNCATE TABLE

数据查询

从数据库中检索数据的过程或命令叫作查询，数据查询是数据库的核心操作。

查询语句的语法如下：

```
SELECT  [ * | expression [ [ AS ] output_name ] [,...] ]

        [ FROM from_item [, ...] ]

        [ WHERE condition** **]

        [ GROUP BY grouping_element [, ...] ]

        [ HAVING condition ]

        [ ORDER BY expression [ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ...] ]

[ LIMIT  count ]
```

一个查询语句通常的执行顺序和处理过程如下：

1. 首先执行FROM子句，会计算所有的行列数据；
2. 如果指定了WHERE子句，所有不满足该条件的行都会被过滤掉；
3. 如果指定了GROUP BY子句或者如果有聚集函数，输出结果会被组合成不同分组，并且计算聚集函数的结果；
4. 如果指定了HAVING子句，会进一步过滤不满足给定条件的分组；
5. SELECT子句将输出指定的列或经过不同表达式计算的列的变体；
6. 如果指定了ORDER BY子句，被返回的行会以指定的顺序排序。如果没有指定ORDER BY子句，系统会以能最快产生行的顺序返回结果；
7. 如果指定了LIMIT或者OFFSET子句，SELECT 语句只返回结果行的一个子集。

FROM子句

FROM子句为SELECT 指定一个或者更多源表。

语法格式：

```
FROM from_item [, ...]
```

其中from_item可以是以下之一：

```
table_name [AS] [alias]
```

```
from_item join_type from_item [ ON join_condition]
```

```
subquery [AS] [alias]
```

参数

1. table_name: 一个现有表或视图的名称。
2. alias: 使用AS关键字为表赋予别名。一个常用的场景是为一个较长的表赋予一个短的名字，使查询语句更易读。当为一个表赋予一个别名后，原表名会被隐藏，对表中列的引用必须以别名来引用。
3. subquery : 子查询指定了一个派生表，在FROM子句后使用子查询必须将其用括号包围，并且必须给它赋予一个别名。更多关于子查询的内容详见6.8小节。
4. join_type: 一个连接表是根据特定的连接类型的规则从两个其他表（真实表或生成表）中派生的表。目前支持内连接、外连接。更多关于连接查询的内容详见6.9小节。

举例说明

假设有weather表，且数据如下：

```
TDM>SELECT * FROM weather;
+-----+ | +-----+ +-----+
| location | | temperature | | time |
+-----+ | +-----+ +-----+
| suzhou   | | 24          | | 2024-03-22 00:00:00.000000 |
+-----+ | +-----+ +-----+
| beijing  | | 18          | | 2024-03-22 00:00:00.000000 |
+-----+ | +-----+ +-----+
```

例1：对weather表进行简单查询。

```
TDM>SELECT * FROM weather;
+-----+ +-----+ +-----+
| location | | temperature | | time |
+-----+ +-----+ +-----+
| suzhou   | | 24          | | 2024-03-22 00:00:00.000000 |
+-----+ +-----+ +-----+
| beijing  | | 18          | | 2024-03-22 00:00:00.000000 |
+-----+ +-----+ +-----+
```

WHERE 子句

语法格式：

WHERE condition

使用说明:

1. condition是任意返回一个boolean类型值的值表达式。任何不满足这个条件的行都会从输出中被消除。

举例说明

假设有weather表，且数据如下:

```
TDM>SELECT * FROM weather;
+-----+|+-----+-----+
|location| |temperature| |time|
+-----+|+-----+-----+
|suzhou  | |24         | |2024-03-22 00:00:00.000000|
+-----+|+-----+-----+
|beijing | |18         | |2024-03-22 00:00:00.000000|
+-----+|+-----+-----+
```

例1: 查询weather表中location列等于'suzhou'的记录。

```
TDM>SELECT * FROM weather WHERE location = 'suzhou';
+-----+|+-----+-----+
|location| |temperature| |time|
+-----+|+-----+-----+
|suzhou  | |24         | |2024-03-22 00:00:00.000000|
+-----+|+-----+-----+
```

例2: 查询weather表中temperature 列值大于20的记录。

```
TDM>SELECT * FROM weather WHERE temperature >20;
+-----+|+-----+-----+
|location| |temperature| |time|
+-----+|+-----+-----+
|suzhou  | |24         | |2024-03-22 00:00:00.000000|
+-----+|+-----+-----+
```

GROUP BY 子句

GROUP BY子句被用来将列中具有相同值的行分组在一起。

语法格式

GROUP BY grouping_element [, ...]

使用说明

1. grouping_element参数可以是一个或一组顺序无关的列，或者包含列的任意表达式。
2. 如果在查询语句中使用聚合函数，则会在每一个分组的所有记录上进行计算，从而为每一个分组产生一个聚合结果值。
3. 通常，如果一个表被分了组，那么没有在GROUP BY中列出的列都不能被引用，除非在聚集表达式中被引用。

举例说明

假设有product表，且数据如下：

```
TDM>SELECT * FROM product;
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
| a           | 20     | 2024-03-26 07:51:20.845469 |
+-----+-----+-----+
| b           | 19     | 2024-03-26 07:51:20.845551 |
+-----+-----+-----+
| c           | 18     | 2024-03-26 07:51:20.845725 |
+-----+-----+-----+
```

例1：对product 表进行查询，根据productname进行分组。

```
TDM>SELECT productname FROM product GROUP BY productname;
+-----+
| productname |
+-----+
| c           |
+-----+
| b           |
+-----+
| a           |
+-----+
```

例2：对product 表进行查询，根据productname进行分组，并计算amount列的和。

```
TDM>SELECT productname,SUM(amount) FROM product GROUP BY productname;
+-----+-----+
| productname | SUM |
+-----+-----+
| c           | 18  |
+-----+-----+
| b           | 19  |
+-----+-----+
| a           | 41  |
+-----+-----+
```

HAVING子句

当使用GROUP BY子句对表分组后，想要筛选出满足指定条件的分组，可以用HAVING子句。

语法格式

```
HAVING condition
```

使用说明

1. 该子句中的condition与 WHERE子句中指定的条件相同，与WHERE不同的是， WHERE会在应用 GROUP BY之前过滤行，而HAVING过滤的是由 GROUP BY分组后的行。

2. condition中引用的每一个列必须无歧义地引用一个在GROUP BY后出现的列，如果要引用未在GROUP BY后出现的列，必须对其使用聚合函数。
3. 如果没有GROUP BY子句，仅使用了HAVING 子句，也会把一个查询转换成一个分组查询。此种情况会把所有行作为一个分组，这和查询中包含聚合函数但没有 GROUP BY子句时的情况相同，所有被选择的行都被认为是一个单一分组，并且SELECT列表和 HAVING子句只能引用聚集函数中的表列。

举例说明

假设有product表，且数据如下：

```
TDM>SELECT * FROM product;
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
| a           | 20     | 2024-03-26 07:51:20.845469 |
+-----+-----+-----+
| b           | 19     | 2024-03-26 07:51:20.845551 |
+-----+-----+-----+
| c           | 18     | 2024-03-26 07:51:20.845725 |
+-----+-----+-----+
```

例1：对product 表进行查询，根据productname进行分组，并筛选出组内amout总和大于40的组。

```
TDM>SELECT productname FROM product GROUP BY productname HAVING SUM(amount) >40;
+-----+
| productname|
+-----+
| a          |
+-----+
```

ORDER BY子句

语法规式

```
[ ORDER BY expression [ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ...] ]
```

使用说明

1. expression 可以是SELECT列表中的输出列或相关的合法的表达式；
2. 当指定多个排序表达式时，依次按照从左至右的顺序排序。
3. 每一个表达式后可以通过设置ASC或DESC关键词来设置排序方向。ASC为升序排序，是默认值，DESC为降序排序。升序会把较小的值放在前面。注意顺序选项只对其紧跟着的列生效，例如ORDER BY x, y DESC表示ORDER BY x ASC, y DESC，而和ORDER BY x DESC, y DESC不同。
4. NULLS FIRST和NULLS LAST选项用来设置NULL值在排序中的行为。NULLS FIRST表示排序时空值被认为比任何非空值都要大，NULLS LAST表示排序时空值被认为比任何非空值都要小。默认情况下为NULLS FIRST。

举例说明

假设有product表，且数据如下：

```
TDM>SELECT * FROM product;
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
| a           | 20     | 2024-03-26 07:51:20.845469 |
+-----+-----+-----+
| b           | 19     | 2024-03-26 07:51:20.845551 |
+-----+-----+-----+
| c           | 18     | 2024-03-26 07:51:20.845725 |
+-----+-----+-----+
```

例1: 查询product表, 并以amount列降序排序。

```
TDM>SELECT * FROM product ORDER BY amount DESC;
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
| a           | 20     | 2024-03-26 07:51:20.845469 |
+-----+-----+-----+
| b           | 19     | 2024-03-26 07:51:20.845551 |
+-----+-----+-----+
| c           | 18     | 2024-03-26 07:51:20.845725 |
+-----+-----+-----+
```

SELECT列表

语法格式

```
SELECT [ * | expression [ [ AS ] output_name ] [, ...] ]
```

使用说明

1. 使用*号将返回FROM子句引用的表生成的所有列。
2. expression 可以是在FROM子句里引用的表中列的实际名字, 也可以是任意列相关的表达式, 除此之外也可以不引用FROM子句中的列的任意常量算术表达式。
3. 如果FROM子句中引用了超过一个表且有相同的列名, 引用时需指定表名。
4. output_name: 通过使用AS可以为输出列指定名字; 如果没有指定AS关键字指定输出列名, 系统会分配一个缺省的列名。对于简单的列引用, 它和被引用列的名字。对于函数调用, 它是函数的名字。对于复杂表达式, 系统会生成一个通用的名字。

举例说明

假设有product表, 且数据如下:

```
TDM>SELECT * FROM product;
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
| a           | 20     | 2024-03-26 07:51:20.845469 |
+-----+-----+-----+
| b           | 19     | 2024-03-26 07:51:20.845551 |
+-----+-----+-----+
| c           | 18     | 2024-03-26 07:51:20.845725 |
+-----+-----+-----+
```

例1：查询product表productname列的数据。

```
TDM>SELECT productname FROM product;
+-----+
| productname |
+-----+
| a           |
+-----+
| a           |
+-----+
| b           |
+-----+
| c           |
+-----+
```

例2：查询product表productname 列以及amount列经过计算后的数据。

```
TDM>SELECT productname ,amount*10 FROM product;
+-----+-----+
| productname | amount * 10 |
+-----+-----+
| a           | 210         |
+-----+-----+
| a           | 200         |
+-----+-----+
| c           | 180         |
+-----+-----+
| b           | 190         |
+-----+-----+
```

例3：查询product表productname 列的数据并为其指定别名。

```
TDM>SELECT productname as prod FROM product;
```

```
+-----+
| prod  |
+-----+
| a     |
+-----+
| a     |
+-----+
| b     |
+-----+
| c     |
+-----+
```

LIMIT子句

LIMIT子句按照指定的偏移量和总量，从查询结果里截取部分数据行。

语法格式

```
LIMIT [offset ,] row_count
```

使用说明

- offset表示开始截取数据行的偏移位置，例如5表示跳过前5行从第6行开始截取。0表示从头开始截取，默认为0。
- row_count表示截取的最大行数。

举例说明

假设有product表，且数据如下：

```
TDM>SELECT * FROM product;
```

```
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
| a           | 20     | 2024-03-26 07:51:20.845469 |
+-----+-----+-----+
| b           | 19     | 2024-03-26 07:51:20.845551 |
+-----+-----+-----+
| c           | 18     | 2024-03-26 07:51:20.845725 |
+-----+-----+-----+
```

例1：查询product表前一条数据。

```
TDM>SELECT * FROM product LIMIT 1;
```

```
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
```

子查询

一个 SELECT-FROM-WHERE 语句称为一个查询块，如果在一个查询块中嵌套一个或多个查询块，我们称这种查询为子查询。子查询会返回一个值（标量子查询）或一个表（表子查询）。它通常采用（SELECT...）的形式嵌套在表达式中。语法结构与查询语法结构一致。

语法格式

```
SELECT  [ * | expression [ [ AS ] output_name ] [, ...] ]

        [ FROM from_item [, ...] ]

        [ WHERE condition** **]

        [ GROUP BY grouping_element [, ...] ]

        [ HAVING condition ]

        [ ORDER BY expression [ ASC | DESC ] [ NULLS { FIRST | LAST }][, ...] ]

        [ LIMIT count ]
```

使用说明

1. 只返回一行一列记录的子查询为标量子查询；
2. 可以返回多行多列的子查询为表子查询，可以用在FROM子句后。

举例说明

假设有product表，且数据如下：

```
TDM>SELECT * FROM product;
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
| a           | 20     | 2024-03-26 07:51:20.845469 |
+-----+-----+-----+
| b           | 19     | 2024-03-26 07:51:20.845551 |
+-----+-----+-----+
| c           | 18     | 2024-03-26 07:51:20.845725 |
+-----+-----+-----+
```

例1：查询出amount列的平均值作为标量子查询，主查询amount列大于最小值的记录。

```
TDM>SELECT * FROM product WHERE amount > (SELECT AVG(amount) FROM product);
+-----+-----+-----+
| productname | amount | time                |
+-----+-----+-----+
| a           | 21     | 2024-03-26 07:51:20.845298 |
+-----+-----+-----+
| a           | 20     | 2024-03-26 07:51:20.845469 |
+-----+-----+-----+
```

例2：查询出productname值为a的记录作为子查询，主查询子查询中amount值大于20的记录。

```
TDM>SELECT * FROM (SELECT * FROM product where productname = 'a') as a1 WHERE  
a1.amount>20;
```

productname	amount	time
a	21	2024-03-26 07:51:20.845298

组合查询

TDM支持对两个查询的结果进行集合操作，目前仅支持并集操作。

语法格式

```
query1 UNION ALL query2
```

使用说明

1. query1和query2可以是任意类型查询，结果不对相同的行去重；
2. 两个查询必须是返回同样数量的列，且对应列的类型是一致的。

举例说明

假设有t1表和t2表，且数据如下：

```
TDM>SELECT * FROM t1;
```

col1	col2	time
a1	21	2024-03-26 07:51:20.000000
a1	20	2024-03-27 07:51:20.000000

```
TDM>SELECT * FROM t2;
```

col1	col2	time
a1	21	2024-03-26 07:51:20.000000
a1	20	2024-03-27 07:51:20.000000

例1：返回对两个表的查询结果的并集。


```
TDM>SELECT * FROM t1 UNION ALL SELECT * FROM t2;
+-----+-----+-----+
| col1   | col2  | time                               |
+-----+-----+-----+
| a1     | 21    | 2024-03-26 07:51:20.000000 |
+-----+-----+-----+
| a1     | 20    | 2024-03-27 07:51:20.000000 |
+-----+-----+-----+
| a1     | 21    | 2024-03-26 00:00:00.000000 |
+-----+-----+-----+
| a1     | 20    | 2024-03-27 07:51:20.000000 |
+-----+-----+-----+
```

连接查询

连接查询是根据特定的连接类型的规则从两个其他表（真实表或生成表）中派生的表。目前支持内连接、外连接。

语法格式

```
T1 { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2 ON boolean_expression
```

使用说明

1. INNER和OUTER对所有连接形式都是可选的。INNER是缺省；LEFT、RIGHT和FULL指示一个外连接，关键字INNER和OUTER可以省略。
2. 连接条件在ON句中指定是最常见的连接条件的形式：它接收一个和WHERE子句里用的一样的布尔值表达式。如果两个分别来自T1和T2的行在ON表达式上运算的结果为真，那么它们就算是匹配的行。
3. 支持的连接类型如下：
4. INNER JOIN：对于 T1 的每一行 R1，生成的连接表都有一行对应 T2 中的每一个满足和 R1 的连接条件的行。
5. LEFT OUTER JOIN：首先，执行一次内连接。然后，为 T1 中每一个无法在连接条件上匹配 T2 中任何一行的行返回一个连接行，该连接行中 T2 的列用空值补齐。因此，生成的连接表里为来自 T1 的每一行都至少包含一行。
6. RIGHT OUTER JOIN：首先，执行一次内连接。然后，为 T2 中每一个无法在连接条件上匹配 T1 中任何一行的行返回一个连接行，该连接行中 T1 的列用空值补齐。因此，生成的连接表里为来自 T2 的每一行都至少包含一行。
7. FULL OUTER JOIN：首先，执行一次内连接。然后，为 T1 中每一个无法在连接条件上匹配 T2 中任何一行的行返回一个连接行，该连接行中 T2 的列用空值补齐。同样，为 T2 中每一个无法在连接条件上匹配 T1 中任何一行的行返回一个连接行，该连接行中 T1 的列用空值补齐。

举例说明

假设有表t1和t2，且它们所包含的数据如下：

```
TDM>SELECT * FROM t1;
+---+-----+-----+
| col| col1  |time                               |
+---+-----+-----+
| 1 | 1     |2024-03-26 08:27:53.505704 |
+---+-----+-----+
| 2 | 2     |2024-03-26 08:27:53.505795 |
+---+-----+-----+
```

```
TDM>SELECT * FROM t2;
+---+-----+-----+
| col | col1 | time |
+---+-----+-----+
| 2 | 1 | 2024-03-26 08:27:53.505704 |
+---+-----+-----+
| 3 | 2 | 2024-03-26 08:27:53.505795 |
+---+-----+-----+
```

例1：对t1表和t2表进行内连接查询。

```
TDM>SELECT t1.col,t2.col FROM t1 INNER JOIN t2 ON t1.col=t2.col;
+-----+-----+
| col | col |
+-----+-----+
| 2 | 2 |
+-----+-----+
```

例2：对t1表和t2表进行左连接查询。

```
TDM>SELECT t1.col,t2.col FROM t1 LEFT JOIN t2 ON t1.col=t2.col;
+-----+-----+
| col | col |
+-----+-----+
| 1 | NULL |
+-----+-----+
| 2 | 2 |
+-----+-----+
```

例3：对t1表和t2表进行右连接查询。

```
TDM>SELECT t1.col,t2.col FROM t1 RIGHT JOIN t2 ON t1.col=t2.col;
+-----+-----+
| col | col |
+-----+-----+
| 2 | 2 |
+-----+-----+
| NULL | 3 |
+-----+-----+
```

窗口查询

TDM支持使用time_bucket函数实现简单的窗口查询，time_bucket函数详介参见8.6.3小节、本节主要介绍实现窗口查询的实例。

使用说明

1. 使用time_bucket降频查询，按分钟、按小时获取数据。
2. 查询一段时间内，统计每十分钟、每小时的平均值、最大值、最小值、总和。

举例说明

例1 假设有访问流量表t1，数据如下：

```
TDM>SELECT * FROM t1;
```

traffic	ts
1	2024-03-26 08:00:00.000000
2	2024-03-26 08:00:01.000000
3	2024-03-26 08:00:02.000000
2	2024-03-26 08:00:03.000000
5	2024-03-26 08:00:04.000000
1	2024-03-26 08:01:01.000000
3	2024-03-26 08:01:02.000000
2	2024-03-26 08:01:03.000000
4	2024-03-26 08:02:00.000000
3	2024-03-26 08:02:03.000000

降频查询（例如， 每分钟的总流量、每小时的平均流量）。

每分钟的总流量

```
TDM>SELECT TIME_BUCKET(60000000,ts) AS bucket, SUM(traffic) FROM t1 GROUP BY bucket
ORDER BY bucket;
```

bucket	sum
2024-03-26 08:00:00.000000	13
2024-03-26 08:01:00.000000	6
2024-03-26 08:02:00.000000	7

每小时的平均流量

```
TDM>SELECT TIME_BUCKET(3600000000,ts) AS bucket, AVG(traffic) FROM t1 GROUP BY bucket
ORDER BY bucket;
```

bucket	avg
2024-03-26 08:00:00.000000	2.6

例2 假设有访问流量表t2，数据如下：

```
TDM>SELECT * FROM t2;
```

traffic	ts
---------	----

1	2024-03-26 08:00:00.000000
+-----+-----+-----+-----+	
2	2024-03-26 08:00:10.000000
+-----+-----+-----+-----+	
3	2024-03-26 08:00:20.000000
+-----+-----+-----+-----+	
4	2024-03-26 08:10:00.000000
+-----+-----+-----+-----+	
5	2024-03-26 08:10:10.000000
+-----+-----+-----+-----+	
6	2024-03-26 08:20:00.000000
+-----+-----+-----+-----+	
7	2024-03-26 08:20:10.000000
+-----+-----+-----+-----+	
8	2024-03-26 08:30:00.000000
+-----+-----+-----+-----+	
2	2024-03-27 08:00:00.000000
+-----+-----+-----+-----+	
3	2024-03-27 08:10:00.000000
+-----+-----+-----+-----+	

统计查询，每十分钟的数据。

```
TDM>SELECT TIME_BUCKET(600000000,ts) AS
bucket,AVG(traffic),SUM(traffic),MAX(traffic),MIN(traffic) FROM t2 GROUP BY bucket ORDER
BY bucket;
```

+-----+-----+-----+-----+				
bucket	avg	sum	max	min
+-----+-----+-----+-----+				
2024-03-26 08:00:00.000000	2	6	3	1
+-----+-----+-----+-----+				
2024-03-26 08:10:00.000000	4.5	9	5	4
+-----+-----+-----+-----+				
2024-03-26 08:20:00.000000	6.5	13	7	6
+-----+-----+-----+-----+				
2024-03-26 08:30:00.000000	8	8	8	8
+-----+-----+-----+-----+				
2024-03-27 08:00:00.000000	2	2	2	2
+-----+-----+-----+-----+				
2024-03-27 08:10:00.000000	3	3	3	3
+-----+-----+-----+-----+				

视图

创建视图

**** ****如果我们有一个复杂的查询，但又不想每次使用时都敲入整个查询。这时我们可以为该查询创建一个视图。

语法格式

```
CREATE VIEW view_name AS query
```

参数

1. view_name: 要创建的视图的名称。
2. query: 提供视图的一个select查询语句。

使用说明

1. 该视图并不会被物理创建，相反，在每一次查询引用该视图时，视图的查询都会被运行。
2. 可以使用show tables命令查看创建的视图列表。

举例说明

创建users表，插入两条数据。

```
TDM>CREATE TABLE users(name string,age int);

Successfully create table

TDM>INSERT INTO users VALUES('jack',24),('john',25);

INSERT 0 2
```

例1: 为对users表的查询创建一个视图，然后查询该视图。

```
TDM>CREATE VIEW myview as SELECT * FROM users where age >20;

Successfully create view

TDM>SELECT * FROM myview ;
+-----+-----+
| name  | age  |
+-----+-----+
| jack  | 24   |
+-----+-----+
| john  | 25   |
+-----+-----+
```

删除视图

TDM支持删除视图。

语法格式

```
DROP VIEW view_name
```

或者使用

```
DROP TABLE view_name
```

举例说明

```
TDM>DROP VIEW mytest11;
```

```
Successfully drop table
```

函数和操作符

逻辑操作符

TDM支持的逻辑操作符包括AND、OR和NOT。他们的运算关系如表8.1，其中AND和OR 交换左右操作数不影响运算结果。

表8.1 逻辑运算符

操作符	描述
boolean AND boolean → boolean	逻辑与
boolean OR boolean → boolean	逻辑或
NOT boolean → boolean	逻辑非

TDM使用三值的逻辑系统，包括TRUE、FALSE和NULL。具体的运算关系如表8.2，表8.3。

表8.2 逻辑运算真值表-1

a	b	a AND b	a OR b
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	NULL	NULL	TRUE
FALSE	FALSE	FALSE	FALSE
FALSE	NULL	FALSE	NULL
NULL	NULL	NULL	NULL

表8.3 逻辑运算真值表-2

a	NOT a
TRUE	FALSE
FALSE	TRUE
NULL	NULL

比较函数和操作符

比较操作符

TDM支持的常用的比较运算符，如表8.4。这些比较操作符适用于所有具有自然排序的内置数据类型，包括数字、字符串、日期/时间类型。

表8.4 比较操作符

操作符	描述
<code>datatype < datatype→boolean</code>	小于
<code>datatype > datatype→boolean</code>	大于
<code>datatype <= datatype→boolean</code>	小于等于
<code>datatype >= datatype→boolean</code>	大于等于
<code>datatype = datatype→boolean</code>	等于
<code>datatype != datatype→boolean</code>	不等于

比较谓词

除以上比较操作符之外，TDM还提供丰富的比较谓词来进行更复杂的比较。如表8.5。

表8.5 比较谓词

谓词 描述 示例
<code>datatype [NOT] BETWEEN datatype AND datatype→boolean</code> 判断值是否在给定范围内，包括范围端点，且适用于任何可以比较的类型。 <code>2 BETWEEN 1 AND 3→true</code> <code>2 NOT BETWEEN 1 AND 3→false</code>
<code>datatype[NOT]BETWEEN SYMMETRC datatype AND datatype→boolean</code> 判断值是否在给定范围内，包括范围端点，端点会自动进行排序，且适用于任何可以比较的类型。 <code>2 BETWEEN SYMMETRIC 3 AND 1→true</code> <code>2 NOT BETWEEN SYMMETRIC 3 AND 1→false</code>
<code>datatype IS NULL→boolean</code> 判断值是否为空 <code>1.5 IS NULL→false</code>
<code>datatype ISNULL→boolean</code> 判断值是否为空（同IS NULL） <code>1.5 IS NULL→false</code>
<code>datatype IS NOT NULL→boolean</code> 判断值是否不为空 <code>'null' IS NOT NULL→true</code>
<code>datatype NOTNULL→boolean</code> 判断值是否不为空（同IS NOT NULL） <code>'null' IS NOT NULL→true</code>

数学操作符和函数

TDM支持丰富的数学操作符和数学函数，如表8.6、表8.7所示。表中可以接受numeric_type 的操作符对所有的 tinyint、smallint、integer、bigint、numeric、float、double类型都可用。可接受 integral_type 的操作符对 tinyint、smallint、integer、bigint类型是可用的。如无特别说明外，操作符返回与其参数相同的数据类型。若运算符中各操作数的数据类型不一致，则可能需要进行类型转换。类型转换大致遵循以下原则：

- 1. 数据类型属于同一类别（如同为数值类型、同为布尔类型等）的操作不需要显式类型转换，而是进行隐式类型转换（包括可能的类型提升）；数据类型分属不同类别的操作数需要通过CAST等函数手动进行显式类型转换（字符串字面量除外）。
- 2. 隐式转换时，按从左到右的顺序，取首个不为字符串字面量的操作数的类型作为隐式类型转换的基础目标类型。
- 3. 字符串字面量与其他类型的参数一同出现时，将尝试隐式类型转换，解析字面量为另一参数的数据类型 的字面量。

表8.6 数学运算符

操作符	描述与示例
numeric_type + numeric_type→numeric_type	加法 2 + 3→5
+ numeric_type→numeric_type	一元加法 + 3.5→3.5
numeric_type - numeric_type→numeric_type	减法 2 - 3→-1
- numeric_type→numeric_type	取负运算 - (-4)→4

操作符	描述与示例
$\text{numeric_type} * \text{numeric_type} \rightarrow \text{numeric_type}$	乘法 $2 * 3 \rightarrow 6$
$\text{numeric_type} / \text{numeric_type} \rightarrow \text{numeric_type}$	除法，对于整型，将结果截断为零 $5.0/2 \rightarrow 2.5000$ $5 / 2 \rightarrow 2$
$\text{integral_type} \% \text{integral_type} \rightarrow \text{integral_type}$	取余运算 $5 \% 4 \rightarrow 1$
$\text{integral_type} \& \text{integral_type} \rightarrow \text{integral_type}$	按位与运算符 $91 \& 15 \rightarrow 11$
integral_type	$\text{integral_type} \rightarrow \text{integral_type}$
$\sim \text{integral_type} \rightarrow \text{integral_type}$	按位取反运算符 $\sim 1 \rightarrow -2$
$\text{integral_type} \# \text{integral_type} \rightarrow \text{integral_type}$	按位异或运算符 $17 \# 5 \rightarrow 20$
$\text{integral_type} \ll \text{integer} \rightarrow \text{integral_type}$	按位左移运算符 $1 \ll 4 \rightarrow 16$
$\text{integral_type} \gg \text{integer} \rightarrow \text{integral_type}$	按位右移运算符 $8 \gg 2 \rightarrow 2$

表8.7数学函数

操作符	描述与示例
<code>acos(numeric_type)→numeric_type</code>	反余弦，结果为弧度，参数范围[-1,1] <code>acos(1) → 0</code>
<code>acosd(numeric_type)→numeric_type</code>	反余弦，结果为度数，参数范围[-1,1] <code>acosd(0.5) → 60</code>
<code>asin(numeric_type)→numeric_type</code>	反正弦，结果为弧度，参数范围[-1,1] <code>asin(1) → 1.57</code>
<code>asind(numeric_type)→numeric_type</code>	反正弦，结果为度数，参数范围[-1,1] <code>asind(0.5) → 30</code>
<code>atan(numeric_type)→numeric_type</code>	反正切，结果为弧度 <code>atan(1) → 0.785</code>
<code>atand(numeric_type)→numeric_type</code>	反正切，结果为度数 <code>atand(1) → 45</code>
<code>exp(numeric_type)→numeric_type</code>	指数 (e 的给定次方) <code>exp(1.0) → 2.718</code>
<code>factorial(integral_type)→numeric_type</code>	阶乘 <code>factorial(5) → 120</code>

操作符	描述与示例
<code>ln(numeric_type)→numeric_type</code>	自然对数 <code>ln(2.0) → 0.693</code>
<code>log(numeric_type)→numeric_type</code>	以10为底的对数 <code>log(100) → 2</code>
<code>log(b numeric_type,x numeric_type)→numeric_type</code>	以 b 为底的 x的对数 <code>log(2.0,64.0) → 6.0</code>
<code>log10(numeric_type)→numeric_type</code>	以10为底的对数 (与 log 相同) <code>log10(1000) → 3</code>
<code>mod(interger,interger)→numeric_type</code>	y/x的余数; 适用于smallint、integer、bigint <code>mod(9, 4) → 1</code>
<code>pi () → numeric_type</code>	π的近似值 <code>pi() → 3.14159</code>
<code>power(numeric_type,numeric_type)→numeric_type</code>	a的b次幂 <code>power(9, 3) → 729</code>
<code>sign(numeric_type)→numeric_type+</code> <code>numeric_type→numeric_type</code>	参数的符号 (-1, 0, 或 +1) <code>sign(-8.4) → -1</code>
<code>sqrt(numeric_type)→numeric_type</code> <code>numeric_type - numeric_type→numeric_type</code>	平方根 <code>sqrt(2) → 1.414</code>
<code>trim_scale(numeric_type)→numeric_type-</code> <code>numeric_type→numeric_type</code>	通过删除尾数部分的零来降低值的刻度(小数位数) <code>trim_scale(8.4100) → 8.41</code>
<code>trunc(numeric_type)→numeric_type</code> <code>numeric_type * numeric_type→numeric_type</code>	截断整数 (向零靠近) <code>trunc(42.8) → 42</code>
<code>trunc(v numeric_type, s integer_type) →</code> <code>numeric_type</code> <code>numeric_type / numeric_type→numeric_type</code>	除法, 对于整型, 将结果截断为零 <code>trunc(42.4382, 2) → 42.43</code>

字符串函数和操作符

本节描述了字符串相关的函数和操作符。除非特别说明，这些函数和操作符声明为接受并返回string类型。

表8.8 SQL字符串函数和操作符

操作符	描述	例子
<code>text text -> text</code>	连接两个字符串	<code>'Post' 'greSQL'</code>

模式匹配

TDM支持使用like操作符进行模式匹配。

语法格式

```
string LIKE pattern [ESCAPE escape-character]

string NOT LIKE pattern [ESCAPE escape-character]
```

使用说明:

1. 如果该string匹配了提供的pattern，那么LIKE表达式返回真（和预期的一样，如果LIKE返回真，那么NOT LIKE表达式返回假，反之亦然。
2. 如果pattern不包含百分号或者下划线，那么该模式只代表它本身的串，这时候LIKE的行为就像等号操作符。
3. 在pattern里使用下划线（_）代表（匹配）任何单个字符； 而一个百分号（%）匹配任何零或多个字符的序列。
4. 如果要匹配下划线或者百分号分别，则在pattern里相应的字符前需要使用逃逸字符。缺省的逃逸字符是反斜线，也可以用ESCAPE子句指定一个不同的逃逸字符。如果要匹配逃逸字符本身，写两个逃逸字符。
5. 由于反斜线在串文本里存在特殊含义且同时是默认的逃逸字符，所以如果要匹配反斜杠常量，需要写4个反斜线。
6. 使用ESCAPE '' 将禁用逃逸机制。

举例说明

```
'abc' LIKE 'abc'      true
'abc' LIKE 'a%'       true
'abc' LIKE '_b_'      true
'abc' LIKE 'c'        false
```

数据类型格式化函数

TDM提供了to_timestamp函数可以将格式化的字符串转换为时间戳。

语法格式

```
to_timestamp ( expr, [,format]) → timestamp
```

使用说明

1. expr只接受字符串类型。
2. format中一些特定的模板序列可以指示函数将输入中对应的部分识别为时间戳的某个字段的数据，当未指定format时，默认format为"YYYY-MM-DD HH24:MI:SS.FF6"。具体的模板序列列表如表8.8所示。

表8.9 用于日期/时间格式化的模板模式

模板序列	说明
YYYY	指代时间戳中的年
MM	指代时间戳中的月，前缀0可以省略
DD	指代时间戳中的日，前缀0可以省略
HH, HH12	指代时间戳中的小时（12小时制），前缀0可以省略
HH24	指代时间戳中的小时（24小时制），前缀0可以省略
MI	指代时间戳中的分钟，前缀0可以省略
SS[.FF{0-9}]	指代时间戳中的秒，前缀0可以省略；FF指代可选的亚秒级数据，最后的数字指代亚秒级输入的最大宽度，前缀0可以省略
- / , . ; : 以及空白字符	分隔符，所有属于分隔符的字符可以互相替代
"text"	双引号内的字符串将原样匹配

1. 3个字符以内的亚秒级输入将被解析为毫秒，否则将被解析为微秒，超过6个字符的亚秒级输入将被丢弃。
2. 输入可以只匹配format的部分前缀，但不满足format的输入将导致报错。例如，对于默认format，"2023/10/15 1:4:0"没有提供亚秒信息，但满足format的年月日时分秒前缀，可以转换为"2023-10-15 01:04:00.000000"；而"2023-10-15T01:04:00"中的T则不满足format，无法转换。

时间/日期函数和操作符

TDM提供了丰富的时间/日期相关的函数和操作符，如表8.9所示：

表8.10 时间函数

函数 描述 示例
<code>current_timestamp</code>→<code>timestamp</code> 当前带有时区的日期和时间（当前事务的开始）。 <code>current_timestamp</code>→2019-12-23 14:39:53.662522
<code>current_timestamp (integer)</code> → <code>timestamp</code> 当前带有时区的日期和时间（当前事务的开始）；有限精度。 <code>current_timestamp(0)</code>→2019-12-23 14:39:53
<code>date_bin (width_us,timestamp,timestamp)</code> → <code>timestamp</code> 将输入的指定的间隔与指定的原点对齐。参见8.6.1小节。 <code>date_bin(60000000, '2020-02-11 15:44:17', '2001-01-01 00:02:30')</code> 结果： 2020-02-11 15:43:30
<code>date_trunc (text, timestamp)</code> → <code>timestamp</code> 截断到指定的精度。参见8.6.2小节。 <code>date_trunc('hour', '2001-02-11 20:38:40')</code> → '2001-02-11 20:00:00'
<code>localtimestamp</code>→<code>timestamp</code> 当前不带时区的日期和时间（当前事务的开始）。 <code>localtimestamp</code>→2019-12-23 14:39:53.662522
<code>localtimestamp (integer)</code> → <code>timestamp</code> 当前不带时区日期和时间（当前事务的开始）；有限精度。 <code>localtimestamp(2)</code>→2019-12-23 14:39:53.66
<code>time_bucket(bucket_width,ts)</code>→ <code>timestamp</code> 将输入的指定的间隔与默认的原点对齐，与<code>date_bin</code>类似，区别在于<code>bucket_width</code>可以是任意更长的间隔。参见8.6.3小节。 <code>time_bucket(60000000, '2020-02-11 15:44:17')</code> 结果： 2020-03-01 15:44:00
<code>now ()</code> → <code>timestamp</code> 当前日期和时间（当前事务的开始）。 <code>now()</code>→2019-12-23 14:39:53.662522-05
<code>timeofday()</code>→<code>text</code> 当前日期和时间（当前语句的开始）。 <code>timeofday()</code> → Mon Jan 06 11:15:15.737617 2025 CST
<code>days_in_month(date)</code> → <code>int</code> 该月所含的天数。参见8.6.4小节。 <code>days_in_month('2020-02-11 15:44:17')</code> → 29
<code>month_normalize(metric, reference_date, days)</code> → <code>double</code> 基于参数<code>reference_date</code>和<code>days</code>对<code>metric</code>进行标准化。参见8.6.5小节。 <code>month_normalize(1000, '2020-02-01', 29)</code> → 1000.0

date_bin

该函数将输入时间按到指定的间隔与指定的原点对齐。

语法格式

```
date_bin(width_us,source,origin)
```

参数

1. width_us: 该参数为bigint类型，单位为微秒，为要使用的时间间隔，目前只支持单位为微秒。比如600000000代表1分钟。该参数必须大于0，否则会抛出错误。
2. source: 该参数为timestamp类型，为要处理的时间。
3. origin: 该参数为timestamp类型，为时间原点，从该原点按照width_us设置的时间间隔对数据分桶。

使用说明

1. 返回值: 新的时间戳，表示时间戳所在的时间桶的起始时间，这个时间会被舍入到最接近的指定时间间隔。
2. 使用场景: 用于将时间戳按照指定时间间隔分桶，用于时间序列数据分析，按照固定时间间隔聚合数据。

举例说明

```
date_bin(600000000, '2020-02-11 15:44:17', '2001-01-01')
```

结果: 2020-02-11 15:44:00

```
date_bin(600000000, '2020-02-11 15:44:17', '2001-01-01 00:02:30')
```

结果: 2020-02-11 15:43:30

假设有表test, 时间列和聚合列数据如下:

ts	value
2024-01-01 10:20:00.000000	25
2024-01-01 10:40:00.000000	30
2024-01-01 11:20:58.000000	28
2024-01-01 11:58:00.000000	32

按小时（3600000000微秒）分桶并计算平均值:

```
TDM>SELECT DATE_BIN(3600000000,ts,'2024-01-01 00:00:00'),AVG(value) FROM test GROUP BY
DATE_BIN ORDER BY date_bin;
```

date_bin	avg
2024-01-01 10:00:00.000000	27.5
2024-01-01 11:00:00.000000	30

date_trunc

该函数将输入时间按指定的精度截断，会将时间舍入到指定单位的开始时间。

语法格式


```
date_trunc(field,source)
```

参数

1. field: 该参数指定以何种精度进行截断。有效的输入值包括micorseconds、milliseconds、second、minute、hour、day、week、month、year。
2. source: 该参数为timestamp类型，为要截断的时间戳。

使用说明

1. 返回值: 新的时间戳，时间部分会被调整到指定粒度。比如，截断到'day'，返回时间戳会是当天开始时间（00:00:00）。
2. 使用场景: 按精度截断时间，适用于时间的标准化、分组和汇总；用于将数据按照特定粒度聚合（如按照分钟、小时汇总事件）。

举例说明

```
date_trunc('hour', '2020-02-11 15:44:17')    → 2020-02-11 15:00:00

date_trunc('year', '2001-02-16 20:38:40')    → 2001-01-01 00:00:00
```

假设有表test，时间列数据如下：

```
ts
-----
2024-01-01 23:00:00.000000
2024-01-01 23:59:00.000000
2024-01-01 23:59:58.000000
2024-01-02 00:01:00.000000
```

按天统计事件结果：

```
TDM>SELECT DATE_TRUNC('day',ts),COUNT(*) FROM test GROUP BY date_trunc ORDER BY
date_trunc;
+-----+-----+
| date_trunc                | count |
+-----+-----+
| 2024-01-01 00:00:00.000000 | 3     |
+-----+-----+
| 2024-02-01 00:00:00.000000 | 1     |
+-----+-----+
```

条件表达式

TDM支持CASE条件表达式，类似软件编程语言中的if/else语句。

语法格式

以下语法为通用表达形式。

```
CASE WHEN condition THEN result
```

```
[WHEN ...]
```

```
[ELSE result]
```

```
END
```

使用说明

1. CASE子句可以用于任何表达式可以出现的地方。
2. 每一个condition是一个返回boolean结果的表达式。如果结果为真，那么CASE表达式的结果就是符合条件的result，并且剩下的CASE表达式不会被执行。如果条件的结果不为真，那么以相同方式执行任何随后的WHEN子句。如果没有WHEN condition为真，那么CASE表达式的值就是在ELSE子句里的result。
3. 如果省略了ELSE子句而且没有条件为真，结果为空。
4. 所有result表达式的数据类型都必须可以转换成单一的输出类型。
5. 以下语法格式为上述通用形式的一个变种。第一个expression会被计算，然后与所有在WHEN子句中的每一个value对比，直到找到一个相等的。如果没有找到匹配的，则返回在ELSE子句中的result。这类似于C里的switch语句。

```
CASE expression
```

```
WHEN value THEN result
```

```
[WHEN ...]
```

```
[ELSE result]
```

```
END
```

举例说明

假设有test表，且数据如下：

```
TDM>SELECT * FROM test;
```

```
+----+
```

```
| a |
```

```
+----+
```

```
| 1 |
```

```
+----+
```

```
| 2 |
```

```
+----+
```

```
| 3 |
```

```
+----+
```

例1：使用CASE语句通用语法格式，对test表的a列进行转换。

```
TDM>SELECT a, CASE WHEN a=1 THEN 'one' WHEN a=2 THEN 'two' ELSE 'other' END FROM test;
+----+-----+
| a  | case |
+----+-----+
| 1  | one  |
+----+-----+
| 2  | two  |
+----+-----+
| 3  | other|
+----+-----+
```

例2：使用CASE语句简单语法格式，对test表的a列进行转换。

```
TDM>SELECT a, CASE a WHEN 1 THEN 'one' WHEN 2 THEN 'two' ELSE 'other'END FROM test;
+----+-----+
| a  | case |
+----+-----+
| 1  | one  |
+----+-----+
| 2  | two  |
+----+-----+
| 3  | other|
+----+-----+
```

聚集函数

聚集函数是将一个输入值的集合计算出一个单一值。TDM提供的聚集函数如表8.10所示。

表8.11 通用聚集函数

函数	描述
sum (smallint) → bigint sum (integer) → bigint sum (bigint) → numeric sum (numeric) → numeric sum (float) → double sum (double) → double	计算非空输入值的总和。
max (type) → same as input type	计算非空输入值的最大值。适用于任何数字、字符串、日期/时间或enum类型，以及inet, interval, money, oid, pg_lsn, tid和任何这些类型的数组。如果没有数据，返回null。
min (type) → same as input type	计算非空输入值的最小值。可用于任何数字、字符串、日期/时间或enum类型，以及inet, interval, money, oid, pg_lsn, tid和任何这些类型的数组。如果没有数据，返回null。
avg(smallint) → numeric avg(integer) → numeric avg(bigint) → numeric avg(numeric) → numeric avg(float) → double avg(double) → double	计算非空输入值的平均值。
count (*) → bigint	计算输入行的数量。
count (colname) → bigint	计算输入值不为空的输入行的数量。

窗口函数

窗口函数与聚合函数类似，但不会将多行合并为单个输出行，而是保留所有原始行并为每行返回一个基于"窗口"的计算结果。

语法格式

```
function_name (expr_list) [ FILTER ( WHERE filter_clause ) ] OVER (window_defination)
```

其中window_defination的语法如下:

```
PARTITION BY expression [, ...] ]
```

```
[ ORDER BY expression [ ASC | DESC | USING operator ] [ NULLS { FIRST | LAST } ] [, ...]  
]
```

```
[ frame_clause ]
```

可选的frame_clause是下列之

```
ROWS frame_start
```

```
ROWS BETWEEN frame_start AND frame_end
```

frame_start 和 frame_end值可定义为:

```
UNBOUNDED PRECEDING
```

```
offset PRECEDING
```

```
CURRENT ROW
```

```
offset FOLLOWING
```

```
UNBOUNDED FOLLOWING
```

使用说明

1. function_name: 窗口函数名称。目前可用作窗口函数的函数如表所示。

表8.12 窗口函数列表

函数	描述
sum ()	参考 聚集函数
max ()	参考 聚集函数
min()	参考 聚集函数
avg()	参考 聚集函数
count()	参考 聚集函数
first()	参考 聚集函数
last()	参考 聚集函数
rank()	返回当前行的排名，必须使用窗口函数语法来调用，也就是说，需要一个OVER子句

1. FILTER 子句: 该子句将在执行窗口计算前先对数据进行筛选，所有的行都会显示，但是不满足条件的行在计算时被视为NULL，不参与计算。

2. PARTITION BY 子句：该子句是窗口函数中最核心的子句之一，它决定了数据如何分组以及窗口函数如何应用。该子句将结果集划分为多个独立的分区，窗口函数在每个分区内单独执行计算。它不像 GROUP BY 那样合并行，而是保留所有原始行。
3. ORDER BY子句：控制分区内行的处理顺序，具体用法同6.5小节。
4. frame_clause：该子句会控制哪些行会被包含在窗口函数的计算中，有两种模式，当使用省略 frame_end的模式时，frame_end默认为current row，当使用between and 模式时，需遵循 frame_end大于frame_start的原则。使用该子句的时候必须指定order by子句，否则返回的行可能不确定。
5. 边界选项详解：

边界表达式	描述
UNBOUNDED PRECEDING	分区的第一行
offset PRECEDING	当前行之前的offset个单位
CURRENT ROW	当前行
offset FOLLOWING	当前行之后的offset个单位
UNBOUNDED FOLLOWING	分区的最后一行

举例说明

创建表并插入数据

```
TDM>CREATE HYPERTABLE sales(customer_id INT AS DIMENSION,amout INT AS MEASURE,ts
TIMESTAMP);

Successfully create table

TDM>INSERT INTO sales VALUES(1,100,'2023-01-01'),(1,200,'2023-01-02'),(1,300,'2023-01-
03'),(2,100,'2023-01-01'),(2,200,'2023-01-02'),(2,300,'2023-01-03');

INSERT 0 6
```

例1：计算每个customer_id的销售总额

```
TDM>SELECT customer_id,amout,ts, SUM(amout) OVER (PARTITION BY customer_id ) FROM SALES;
+-----+-----+-----+-----+
| customer_id | amount | ts                | sum  |
+-----+-----+-----+-----+
| 1           | 100    | 2023-01-01 00:00:00.000000 | 600  |
+-----+-----+-----+-----+
| 1           | 200    | 2023-01-02 00:00:00.000000 | 600  |
+-----+-----+-----+-----+
| 1           | 300    | 2023-01-03 00:00:00.000000 | 600  |
+-----+-----+-----+-----+
| 2           | 100    | 2023-01-01 00:00:00.000000 | 600  |
+-----+-----+-----+-----+
| 2           | 200    | 2023-01-02 00:00:00.000000 | 600  |
+-----+-----+-----+-----+
| 2           | 300    | 2023-01-03 00:00:00.000000 | 600  |
+-----+-----+-----+-----+
```

例2：计算每日及其前一天和后一天的销售总额。

```
TDM>SELECT customer_id,amout,ts, SUM(amout) OVER (PARTITION BY customer_id ORDER BY  
customer_id, AMOUT ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING ) FROM SALES;
```

customer_id	amount	ts	sum
1	100	2023-01-01 00:00:00.000000	300
1	200	2023-01-02 00:00:00.000000	600
1	300	2023-01-03 00:00:00.000000	500
2	100	2023-01-01 00:00:00.000000	300
2	200	2023-01-02 00:00:00.000000	600
2	300	2023-01-03 00:00:00.000000	500

类型转换函数

除了8.3小节提到的隐式类型转换情况，TDM还提供了显式转换类型函数CAST或者“::”。

语法格式

```
CAST (expr AS type)
```

```
expr :: type
```

使用说明

1. 尝试将expr转换为TYPE类型。
2. 转换到整数类型时，如果expr的整数值在TYPE值域之外，则操作将报错。
3. TIMESTAMP与字符串类型之间的转换使用默认format "YYYY-MM-DD HH24:MI:SS.FF6"。

举例说明

```
CAST ('1' AS 1) → 1
```

```
CAST (true AS string) → 'true'
```

例1 目前查询当前时间的前100分钟的数据，需要将时间转换为微秒级整数进行数学运算后再转回时间戳：

```
TDM>SELECT cola,colb,ts FROM table1 WHERE ts >=(NOW()::i64-100*60*1000000)::TIMESTAMP  
LIMIT 100;
```

行比较

本节描述几个特殊的结构，用于在一组值之间进行多重比较。

IN

语法格式

```
expression IN ( value** **[, ...])
```

使用说明

1. IN右边括号内为标量列表。如果expression的值等于括号内任意一个值，则表达式的结果为真，反之为假。
2. 如果expression的值为空值，或者括号内没有与expression相等的值且至少包含一个空值，那么IN表达式的结果将为空值，而不是假。

举例说明

```
1 in (1,2 ,3)      →   true  
  
6 in (1,2, 3)      →   false  
  
null in (1,2,3)    →   null
```

NOT IN

语法格式

```
expression NOT IN ( value** **[, ...])
```

使用说明

1. IN右边括号内为标量列表。如果expression的值不等于括号内任意一个值，则表达式的结果为真，反之为假。
2. 如果expression的值为空值，或者括号内没有与expression相等的值且至少包含一个空值，那么NOT IN表达式的结果将为空值，而不是假。

举例说明

```
1 not in (1,2, 3)   →   false  
  
6 not in (1,2 ,3)   →   true  
  
not null in (1,2,3) →   null
```

时序函数

TDM支持一系列时序函数。通过使用这些函数能高效处理、聚合和分析大量时序数据。

time_bucket

该函数将输入时间按到指定的间隔与默认的原点（2000-01-03 00:00:00）对齐，允许将hypertable中的数据按照时间间隔聚合到时间桶中。

语法格式

```
time_bucket(bucket_width,ts)
```

参数

1. bucket_width: 该参数为bigint类型，单位为微秒，为要使用的时间间隔，目前只支持单位为微秒。需要设置10分钟、1小时、1天、1个月等时间长度为间隔，换算为对应的微秒值，比如60000000代表1分钟。该参数必须大于0，否则会抛出错误。
2. ts: 该参数为要换算的timestamp类型的值表达式。

使用说明

1. 返回值: 新的时间戳，表示时间戳所在桶的起始时间，这个时间会被舍入到最接近的指定时间间隔。
2. 按照指定时间间隔分桶，进行时间序列数据分析和聚合通常和GROUP BY结合使用以聚合数据。例如，可以计算bucket中值的平均值、最大值、最小值、总和。
3. 时间原点: 2000-01-03 00:00:00，从该原点按照bucket_width设置的时间间隔对数据分桶。

举例说明

例1: 单个时间值上运行time_bucket，可以看到该时间位于哪个桶:

```
time_bucket(60000000, '2020-02-11 15:44:17')
```

结果: 2020-02-11 15:44:00

```
time_bucket(3600000000, '2020-02-11 15:44:17')
```

结果: 2020-03-01 15:00:00

例2: 假设有表test, 时间列和聚合列数据如下:

ts	value
2024-01-01 10:20:00.000000	25
2024-01-01 10:40:00.000000	30
2024-01-01 11:20:58.000000	28
2024-01-01 11:58:00.000000	32

按小时（3600000000微秒）分桶并计算最大值:

```
TDM>SELECT TIME_BUCKET(3600000000,ts),MAX(value) FROM test GROUP BY time_bucket ORDER BY time_bucket;
```

time_bucket	max
2024-01-01 10:00:00.000000	30
2024-02-01 11:00:00.000000	32

days_in_month

该函数根据输入时间获取当月所含天数。

语法格式

```
day_in_month(fource)
```

参数

1. source: 该参数为timestamp类型，为要处理的时间。

举例说明

```
days_in_month('2020-02-11 15:44:17')
```

结果: 29

```
days_in_month('2020-03-04')
```

结果: 31

```
days_in_month(null)
```

结果: null

month_normalize

该函数基于参数reference_date和days对metric进行标准化，这个过程通常用于将不同天数的时间段（如每个月）的指标值转换为基于某个标准天数（如全年度天数）的指标值。

语法格式

```
month_normalize(metric, reference_date, [days])
```

参数

1. metric: 该参数为double类型，需要标准化的指标值。
2. reference_date: 该参数为timestamp类型，作为标准化参考点的时间戳，根据与此日期对应的月份中的天数进行标准化。
3. days: 可选参数，该参数为double类型，用于标准化的天数，未提供使用默认值365.25/12。

使用说明

1. 返回值: double类型，标准化后的指标值。
2. 计算逻辑: 函数结果=metric*days/reference_date该月的天数，如果有一个参数为NULL 则结果为NULL，days未指定时使用默认值365.25/12。
3. 使用场景: 时间序列分析，比较不同长度月份的指标（如2月和3月）；月度聚合，聚合或比较月度期间的指标，对数据标准化，以避免月长带来的偏差等。

举例说明

```
month_normalize(1000, '2020-02-01')
```

结果: 1049.5689655172414

```
month_normalize(1000, '2020-02-01', 29)
```

结果: 1000

```
month_normalize(1000, null)
```

结果: null

first

该函数将根据某一列排序后，返回该列排序第一行对应的另一指定列值。

语法格式

```
first(value, order_column)
```

参数

- 1. value: 将要返回该列值。
- 2. order_column:根据该列进行排序。

举例说明

假设有表test, 时间列和聚合列数据如下:

ts	value	device
2024-01-01 10:20:00.000000	25	a
2024-01-01 10:40:00.000000	30	a
2024-01-01 10:20:00.000000	28	b
2024-01-01 10:40:00.000000	32	b

例1: 返回按device列分组时间最早的value列值。

```
TDM>SELECT device, FIRST (value,ts) FROM test GROUP BY device;
+-----+-----+
| device |first  |
+-----+-----+
| a      | 25    |
+-----+-----+
| b      | 28    |
+-----+-----+
```

last

该函数将根据某一列排序后，返回该列排序第一行对应的另一指定列值。

语法格式

```
last(value, order_column)
```

参数

- 1. value: 将要返回该列值。
- 2. order_column:根据该列进行排序。

举例说明

假设有表test, 时间列和聚合列数据如下:

ts	value	device
2024-01-01 10:20:00.000000	25	a
2024-01-01 10:40:00.000000	30	a
2024-01-01 10:20:00.000000	28	b
2024-01-01 10:40:00.000000	32	b

例1：返回按device列分组时间最晚的value列值。

```
TDM>SELECT device, LAST(value,ts) FROM test GROUP BY device;
```

device	last
b	32
a	30

结果集缓存

TDM支持支持以表为粒度进行数据库结果集缓存的独立配置，即允许用户按需为不同数据表单独设置缓存开关：例如，可对表A启用结果集缓存，而对表B保持关闭状态。

使用结果集缓存功能前，须先在数据库层面启用结果集缓存全局配置，方可进一步针对特定数据表开启表级缓存。参照服务器配置，开启数据库的结果集缓存功能。

建表设置缓存

在建表时可以指定表的结果集缓存配置。

语法格式

```
CREATE {TABLE | HYPERTABLE} table_name(...) [RESULT_CACHE {DEFAULT | ENABLE | DISABLE}];
```

参数

1. DEFAULT: 采用系统默认的表结果集缓存配置，目前默认是enable。
2. ENABLE: 显式开启该表的结果集缓存。
3. DISABLE: 显式关闭该表的结果集缓存，后续对该表的查询结果不会被缓存。

说明

1. [RESULT_CACHE ...] 整体可以不写，即默认表缓存状态是invalid的，不开启状态。

举例说明

例1：创建表时指定表缓存配置。

```
TDM>CREATE TABLE test_result_cache(c1 int) RESULT_CACHE DEFAULT;
Successfully create table

TDM>CREATE TABLE test_result_cache1(c1 int) RESULT_CACHE ENABLE;
Successfully create table

TDM>CREATE TABLE test_result_cache2(c1 int) RESULT_CACHE DISABLE;
Successfully create table

TDM>CREATE TABLE test_result_cache3(c1 int);
Successfully create table
```

修改表缓存

已有表可以通过alter修改表的结果集缓存配置。

语法格式

```
ALTER table_name ALTER RESULT_CACHE {DEFAULT | ENABLE | DISABLE};
```

或者直接删除表缓存配置

语法格式

```
ALTER table_name DROP RESULT_CACHE;
```

参数

1. DEFAULT: 采用系统默认的表结果集缓存配置，目前默认是enable。
2. ENABLE: 显式开启该表的结果集缓存。
3. DISABLE: 显式关闭该表的结果集缓存，后续对该表的查询结果不会被缓存。

说明

1. DROP RESULT_CACHE，即默认表缓存状态是invalid的，不开启状态。

举例说明

例1：修改表缓存配置。

```
TDM>ALTER TABLE test_result_cache ALTER RESULT_CACHE DEFAULT;
Successfully alter table

TDM>ALTER TABLE test_result_cache1 ALTER RESULT_CACHE ENABLE;
Successfully alter table

TDM>ALTER TABLE test_result_cache2 ALTER RESULT_CACHE DISABLE;
Successfully alter table

TDM>ALTER TABLE test_result_cache3 DROP RESULT_CACHE;
Successfully alter table
```

查看表缓存

支持查看已有表的结果集缓存配置。

语法格式

```
SHOW TABLE STATUS [IN database_name | LIKE pattern];
```

说明

1. [...] 整体可以不写，即查看当前连接数据库中表缓存配置。
2. IN database_name 查看指定数据库中表缓存配置。
3. LIKE pattern查看当前数据库中表名符合该模式匹配的表缓存配置。

举例说明

例1：查看当前数据库中表名含'test_result_cache'的表缓存配置。

```
TDM>SHOW TABLE STATUS LIKE 'test_result_cache%';
Name                                | Create_options
-----+-----
test_result_cache                   | Eable
test_result_cache1                  | Eable
test_result_cache2                   | Disable
test_result_cache3                   | Invalid
```

发布订阅功能

TDM支持发布订阅功能（目前普通表不支持），支持主题创建、数据订阅、消费者消费、主题删除全流程管理，实现数据的生产、订阅、消费与资源销毁，满足流式数据分发与解耦场景。

创建主题

基于已有数据源表，创建订阅主题，实现数据自动流入。

语法格式

```
CREATE TOPIC topic_name PARTITION BY DIMENSION(0) AS SELECT { * | column_name.... } FROM table_name [WHERE condition]
```

参数

1. topic_name: 将要创建的主题名称。
2. table_name: 要订阅的数据表。

说明

1. 可以订阅数据表的指定几列，也可以用*指定订阅全部列。
2. 主题是全局唯一的。
3. 只有在创建主题后插入的数据才可以被订阅到。
4. 只订阅插入和导入的数据，更新和删除的数据不会被订阅到。
5. 被订阅的表不可以做alter或drop 操作。

举例说明

假设有表test, 时间列和聚合列数据如下：

ts	value	device
2024-01-01 10:20:00.000000	25	a
2024-01-01 10:40:00.000000	30	a
2024-01-01 10:20:00.000000	28	b
2024-01-01 10:40:00.000000	32	b

例1：创建主题订阅test表的数据。

```
TDM>CREATE TOPIC subtest PARTITION BY dimension(0) AS SELECT * FROM test;
```

```
Successfully create topic
```

删除主题

删除已存在的主题，释放资源，主题删除后无法恢复。

语法格式

```
DROP TOPIC topic_name
```

参数

1. topic_name: 将要删除的主题名称。

举例说明

例1: 删除主题。

```
TDM>DROP TOPIC subtest;  
  
Successfully drop topic
```

订阅主题并消费

指定消费者从目标主题拉取数据，完成订阅消费。

语法格式

```
FETCH DATA FROM consumer consumer_name topic topic_name  
  
[Limit <batch_size>]
```

参数

1. consumer_name: 创建的消费者名称。
2. topic_name: 订阅的主题名称。
3. batch_size: 设置单次消费的数据条数。

说明

1. 在第一次拉取数据时自动创建消费者，并自动订阅主题。
2. 数据消费的位点自动提交并记录，下次消费会自动从上一次消费位置开始消费数据。
3. 不设置batch_size时默认会拉取全部数据，但最多不超过10万条。

举例说明

例1: 创建时序表testsubscribe ,创建订阅该表的主题并拉取数据 。

创建数据表

```
TDM>CREATE HYPERTABLE testsubscribe (device string as dimension,value int as measure,ts  
timestamp);  
  
Successfully create table
```

创建主题

```
TDM>CREATE TOPIC subtestsubscribe PARTITION BY dimension(0) AS SELECT * FROM  
testsubscribe;  
  
Successfully create topic
```

向源表插入数据

```
TDM>INSERT INTO testsubscribe VALUES('DEVICE0',24,'2024-03-22 00:00:00');  
  
INSERT 0 1
```


消费数据

```
TDM>FETCH DATA FROM CONSUMER consumer TOPIC subtestsubscribe;
```

device	value	ts
DEVICE0	24	2024-03-22 00:00:00.000000

地理数据类型支持

TDM支持存储地理信息系统对象以及对他们的简单操作。

OGC Geometry

开放地理空间联盟（OGC）定义了简单功能访问标准（SFA），以提供地理空间数据模型。它定义了几何的基本空间类型，以及操纵和转换几何值以执行空间分析任务的操作。TDM通过定义称为Geometry的数据类型来实现OGC的简单要素模型

WKT和EWKT

OGC SFA规范定义了用于表示外部使用几何值的标准格式。WKT包含有关定义对象的类型和坐标的信息。WKT提供空间数据的标准字符表示形式。以下是空间对象的WKT表示形式示例：

```
POINT(0 0)

LINESTRING(0 0,1 1,2 2)

POLYGON((0 0,4 0,4 4,0 4,0 0),(1 1,2 1,2e 2,1 2,1 1))
```

TDM定义了扩展的EWKT格式。它们提供坐标支持，并包括SRID信息。创建插入数据库的空间对象时需要提供SRID（默认为0）。

以下是空间对象的EWKT表示形式示例：

```
POINT(0 0)

SRID=5432;POINT(0 0)

LINESTRING(0 0,1 1,2 2)

SRID=4326;LINESTRING(0 0,1 1,2 2)

POLYGON((0 0,4 0,4 4,0 4,0 0),(1 1,2 1,2 2,1 2,1 1))

SRID=4326;POLYGON((0 0,4 0,4 4,0 4,0 0),(1 1,2 1,2 2,1 2,1 1))
```

Point

Point类型代表二维空间中的一个位置。以下例子将展示如何定义该类型并且插入数据。

举例说明

```
TDM>CREATE TABLE gis_point (location GEOMETRY(POINT));

Successfully create table

TDM>INSERT INTO gis_point VALUES('SRID=4326;POINT(-110 30)');

INSERT 0 1

TDM>INSERT INTO gis_point VALUES('POINT(-110 30)');
```

```
INSERT 0 1
```

```
TDM>SELECT * FROM gis_point;
+-----+
| location          |
+-----+
| SRID=4326;POINT (-110 30) |
+-----+
| SRID=0;POINT (-110 30)    |
+-----+
```

LineString

线串是由连续的线段序列形成的一维直线。每条线段由两个点定义，其中一条线段的终点构成下一条线段的起点。TDM允许使用单点线串。线串可以相互交叉（自相交），也可以是起点和终点相同。以下例子将展示如何定义该类型并且插入数据。

举例说明

```
TDM>CREATE TABLE gis_linestring (location GEOMETRY(LINESTRING));

Successfully create table

TDM>INSERT INTO gis_linestring VALUES

('SRID=4326;LINESTRING(117.166667 23.000001,121.500001 23.000001)');

INSERT 0 1
```

Polygon

多边形是由外部边界和零个或多个内部边界分隔的二维平面区域。以下例子将展示如何定义该类型并且插入数据。

举例说明

```
TDM>CREATE TABLE gis_polygon (location GEOMETRY(POLYGON));

Successfully create table

TDM>INSERT INTO gis_polygon VALUES('SRID=4326; POLYGON ((117.166667 23.000001,121.500001 23.000001,121.500001 27.000001,117.166667 27.000001,117.166667 23.000001))');

INSERT 0 1
```

ST_GeomFromText

该函数从已知的文本表示（EWKT）构造Geometry对象。

语法格式

```
ST_GeomFromText(EWKT,SRID)
```

参数

1. EWKT: 该参数为EWKT文本表示。
2. SRID: 该参数为可选参数, 是整型表示的SRID, 当省略时默认为0。

举例说明

```
TDM>SELECT ST_GeomFromText('POINT(121.07983 26.41713)',4326);
+-----+
| st_geomfromtext |
+-----+
| SRID=4326;POINT (121.07983 26.41713) |
+-----+
```

ST_Polygon

该函数返回给定LineString构建的多边形, 并根据SRID设置空间参考系统。

语法格式

```
ST_Polygon(lineString,SRID)
```

参数

1. lineString: 该参数为geometry(LINESTRING) 类型。
2. SRID: 该参数用于设置空间参考系统。

举例说明

```
TDM>SELECT ST_Polygon('LINESTRING(117 23,121 23,121 27,117 27,117 23)::geometry,4326);
+-----+
| st_polygon |
+-----+
| SRID=4326;POLYGON ((117 23, 121 23, 121 27, 117 27, 117 23)) |
+-----+
```

ST_DWithin

该函数判断两个几何体之间的距离是否在给定距离内, 如果是则返回true, 否则返回false。

语法格式

```
ST_DWithin(geometry1,geometry2,distance)
```

参数

1. geometry1: 该参数为geometry类型: Point、LineString或者Polygon类型。
2. geometry2: 同geometry1。
3. distance: 该参数为double类型, 代表距离参数。

举例说明

```
TDM>SELECT ST_DWithin('SRID=4326;POINT(120 26)','SRID=4326;POINT(120 27)',0.1);
+-----+
| st_dwithin |
+-----+
| FALSE      |
+-----+
```

ST_Contains

该函数用于判断两个几何图像的包含关系。

语法格式

```
ST_Contains(geometryA,geometryB)
```

说明

1. geometryA和geometryB为任意geometry几何类型。
2. 如果几何图像A包含几何图像B，则返回TRUE。当且仅当B的点都位于A内部时，A包含B，并且A和B的内部至少有一个共同点。包含关系是自反的，每个图像都包含其自身。

举例说明

```
TDM>SELECT ST_Contains(ST_Polygon('LINESTRING(117 23,121 23,121 27,117 27,117
23) '::geometry,4326), 'SRID=4326;POINT(120.66 26.344)');
+-----+
| st_contains |
+-----+
| TRUE        |
+-----+
```

geometry数据类型及空间函数使用实例

使用Point、LineString 和 Polygon 表示地理空间数据，使用ST_Contains、ST_DWithin、ST_GeomFromText 和 ST_Polygon 等函数进行空间数据的查询和分析。

这些数据类型和函数广泛应用于地理信息系统（GIS）、地图服务、位置跟踪、空间分析等领域，能够帮助用户实现复杂的地理空间数据处理和分析任务。

一个简单的使用实例：

创建包含几何类型的表并存入数据：

```
TDM>CREATE HYPERTABLE places (name TEXT AS MEASURE,location GEOMETRY(POINT) AS MEASURE
NULL,`ts` TIMESTAMP DEFAULT '2024-08-07 00:00:00');
```

Successfully create table

```
TDM>CREATE HYPERTABLE routes (name TEXT AS MEASURE,path geometry(LINESTRING) AS MEASURE
NULL,`ts` TIMESTAMP DEFAULT '2024-08-07 00:00:00');
```

Successfully create table

```
TDM>CREATE HYPERTABLE parks (name TEXT AS MEASURE,boundary geometry(POLYGON) AS MEASURE
NULL,`ts` TIMESTAMP DEFAULT '2024-08-07 00:00:00');
```

Successfully create table

插入Point数据

```
TDM>INSERT INTO places (name,location) VALUES('test01','POINT(121.07983 26.41713)');
```

INSERT 0 1

```
TDM>INSERT INTO places (name,location) VALUES('test02','POINT(-7.1 4.8)');
```

INSERT 0 1

```
TDM>INSERT INTO places (name,location) VALUES('test03','POINT(-7.4 4.9)');
```

INSERT 0 1

插入LineString数据

```
TDM>INSERT INTO routes (name, path) VALUES('Route 1', 'LINESTRING(-7.1 4.8,-7.5 4.1)');
```

INSERT 0 1

插入Polygon数据

```
TDM>INSERT INTO parks (name, boundary) VALUES('Central Park', 'POLYGON((-7.1 4.8, -7.5
4.8, -7.5 4.1, -7.1 4.1, -7.1 4.8))');
```

INSERT 0 1

查询所有地点:

```
TDM>SELECT name, location FROM places;
+-----+-----+
| name  | location                               |
+-----+-----+
| test01 | SRID=0;POINT (121.07983 26.41713) |
+-----+-----+
| test02 | SRID=0;POINT (-7.1 4.8)           |
+-----+-----+
| test03 | SRID=0;POINT (-7.4 4.9)           |
+-----+-----+
```

查询指定地点周围0.1内的所有的点:

```
TDM>SELECT name, location FROM places WHERE ST_DWithin('POINT(-7.1 4.8)', location,
0.1);
+-----+-----+
| name   | location                               |
+-----+-----+
| test02 | SRID=0;POINT (-7.1 4.8)              |
+-----+-----+
```

或者查询语句如下:

```
TDM>SELECT name, location FROM places WHERE ST_DWithin('POINT(-7.1
4.8)::GEOMETRY(POINT), location,0.1);
+-----+-----+
| name   | location                               |
+-----+-----+
| test02 | SRID=0;POINT (-7.1 4.8)              |
+-----+-----+
```

判断一个点是否在多边形内:

```
TDM>SELECT ST_Contains(ST_GeomFromText('POLYGON((-7.1 4.8, -7.5 4.8, -7.5 4.1, -7.1 4.1,
-7.1 4.8))'),ST_GeomFromText('POINT(-7.3 4.6)'));
+-----+-----+
| st_contains |
+-----+-----+
| t           |
+-----+-----+
```

查询places表中位于多边形区域内point数据:

```
TDM>SELECT name, location FROM places WHERE ST_Contains(ST_GeomFromText('POLYGON((-7.1
4.8, -7.5 5, -7.5 4.1, -7.1 4.1, -7.1 4.8))'), location);
+-----+-----+
| name   | location                               |
+-----+-----+
| test03 | SRID=0;POINT (-7.4 4.9)              |
+-----+-----+
```

预编译语句

PREPARE

TDM支持PREPARE创建一个预备语句。

语法格式：

```
PREPARE name AS statement
```

参数

1. name：给这个特定的预备语句命名。在一个会话中必须唯一。
2. statement：任何SELECT、INSERT、UPDATE、DELETE语句。

使用说明

1. 预备语句暂时不支持接受参数，在编写预备语言时sql语句中为实际值。
2. 预备语句只在当前数据库会话期间存在。当会话结束，预备语句消失。每个客户端都可以创建需要的预备语句。
3. 预备语句被用来优化性能。在一个会话要执行大量类似语句时，或者复杂语句（难于规划或者重写），存在性能优势，或者性能差异特别明显。简单语句，并且执行开销较大的语句，使用预备语句没有显著优势。

举例说明

例如在多表join时预编译查询语句：

```
TDM>PREPARE pre_select AS select first_name||last_name,department_name from employees  
inner join departments on employees.department_id = departments.department_id;
```

```
PREPARE
```

EXECUTE

TDM支持使用EXECUTE执行一个预备语句。

语法格式：

```
EXECUTE name
```

参数

1. name：要执行的预备语句的名称。

使用说明

1. 因为预备语句只在当前数据库会话期间存在。该预备语句必须在当前会话中由一个更早执行的PREPARE所创建。

举例说明


```
TDM>EXECUTE pre_select;
+-----+-----+-----+
| first_name | last_name      | department_name |
+-----+-----+-----+
| Jane       | Smith          | HR              |
+-----+-----+-----+
| John       | Doe            | Sales           |
+-----+-----+-----+
```

DEALLOCATE

TDM支持使用DEALLOCATE释放一个预备语句。

语法格式：

```
DEALLOCATE name
```

参数

1. name：要释放的预备语句的名称。

使用说明

1. 如果不显式地释放一个预备语句，会话结束时会自动释放它。

举例说明

```
TDM>deallocate pre_select;
DEALLOCATE
```

备份与恢复

TDM 提供两类数据导出能力，分别适配全量备份迁移、按需报表提取两种场景：EXPORT/IMPORT 命令：支持整库、单表级别的完整数据导出与恢复，主要用于离线备份、跨版本环境数据迁移，可同时导出表元数据结构与业务时序数据；SELECT ... OUTFILE 语法：支持将自定义查询结果导出为 CSV 文本文件，适用于业务筛选、数据分析、报表导出场景，仅导出查询行数据，不含表结构元信息。

备份

使用export命令导出表或整个数据库。

语法格式

```
EXPORT [DATABASE | TABLE] object_name [IGNORE[SCHEMA | ALL] TO 'output file path' TEMP 'temp dir'
```

参数

1. database | table: 该选项用于设定导出指定单个表还是导出单个数据库。
2. object_name: 要导出的数据库或者表的名称。
3. ignore: 设置该参数可覆盖已存在输出文件。
4. schema | all: 设置该参数为schema则只导出数据库或者表的元数据部分，设置该参数为all则将元数据和数据一起导出。
5. output file path: 导出文件存放路径。
6. temp dir: 用于存储临时过程文件。

举例说明

例1. 假设有表cpu，将cpu表定义及数据全部导入到文件中。

```
TDM>EXPORT TABLE cpu ALL TO '/root/tb_export' TEMP '/root';  
  
Export 1000 rows INTO file root/tb_export using 66ms
```

例1. 假设有数据库dbtest，将dbtest数据库中的所有表及数据全部导入文件中。

```
TDM>EXPORT DATABASE dbtest ALL TO '/root/db_export' TEMP '/root';  
  
Export 2000 rows INTO file root/db_export using 99.6ms
```

使用SELECT ... OUTFILE 语法导出 CSV 文本文件。

语法格式

```
select *|column_name... from tablename into outfile 'output file path'  
  
[FIELDS TERMINATED BY 'string ']  
  
[LINES TERMINATED BY 'string ']  
  
[timestamp as bigint]
```

参数

1. select *|column_name... from tablename: 标准的select语句，指定将要导出的数据行。
2. FIELDS TERMINATED BY 'string': 指定字段值的分隔符，不指定时默认是逗号','。
3. LINES TERMINATED BY 'string': 指定数据行的分隔符，不指定时默认分隔符为'\n'。
4. timestamp as bigint:指定该语句，则会自动把时间列转换为整型，大幅度提升导出速度，一般建议加上该选项。

举例说明

例1.假设现有表cpu，将cpu表数据全部导入到文件中。

```
TDM>select * from cpu into outfile '/root/cpudata.csv' fields terminated by ',' lines
terminated by '\n' timestamp as bigint;
SELECT 100 rows INTO file /root/cpudata.csv using 184.33ms
```

恢复

通过将导出的数据文件重新导入到数据库中来恢复数据。

语法格式

```
IMPORT FROM 'input file path' TEMP 'temp dir'
```

参数

1. input file path:导入文件存放路径。
2. temp dir:用于存储临时过程文件。

使用说明

1. 如果存在同名表或数据库会导入失败。
2. 目前不支持postgres库的导入。

举例说明

例1.将前文中导出的tb_export文件导入到数据库中。

```
TDM>IMPORT FROM '/root/tb_export.csv' TEMP '/root';

COPY 1000 rows of 1 files using 4.1ms
```

例2.将前文中导出的db_export文件导入到数据库中。

```
TDM>IMPORT FROM '/root/db_export.csv' TEMP '/root';

COPY 2000rows of 1 files using 4.1ms
```

应用程序连接示例

JAVA

JDBC简介

JDBC (Java Database Connectivity) 是 Java 应用程序与数据库的接口规范, 旨在让各数据库开发商为 Java 程序员提供标准的数据库应用程序编程接口 (API)。JDBC 定义了一个跨数据库、跨平台的通用 SQL 数据库 API。TDM 兼容 PostgreSQL 的 JDBC 驱动程序 (要求版本 42.7.2 及以上), 它是一个能够支持基本 SQL 功能的通用应用程序编程接口, 支持一般的 SQL 数据库访问。通过 JDBC 驱动程序, 用户可以在应用程序中实现对 TDM 数据库的连接与访问, JDBC 驱动程序的主要功能包括:

- 建立与 TDM 数据库的连接;
- 转接发送 SQL 语句到数据库;
- 处理并返回语句执行结果。

使用前准备

使用 Java 连接 TDM 数据库前, 需要具备以下条件:

- 已安装 Java 1.8 或以上版本运行时环境和 Maven 3.6 或以上版本。
- 项目中已包含了对 PostgreSQL 的 JDBC 驱动 42.7.2 版本及以上的引用。如果你使用 Maven, 可以添加以下依赖到你的 pom.xml 文件中, 版本可根据实际情况更改。

```
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.2</version>
</dependency>
```

应用示例

下面用一个具体的编程实例来展示利用 JDBC 驱动程序进行数据库操作的基本步骤:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
import java.sql.PreparedStatement;

public class JDBCExample {
    public static void main(String[] args) {

        // 定义连接TDM的URL连接串, 目前需要设置sslmode为disable, 端口号为TDM_ROUTER的端口号, 默认为25432。
        String url = "jdbc:postgresql://localhost:25432/postgres?sslmode=disable";

        //定义用户名, 目前仅支持postgres用户
        String user = "postgres";

        //定义用户口令, 目前默认口令为dameng
        String password = "dameng";

        try {
```

```

// 加载并注册JDBC驱动
Class.forName("org.postgresql.Driver");

// 建立数据库连接
Connection conn=DriverManager.getConnection(url, user, password);

// 创建Statement对象来执行SQL语句
Statement stmt = conn.createStatement();

//创建PreparedStatement对象来执行语句
PreparedStatement pstmt=null;

//创建表
stmt.execute("CREATE TABLE mytable (column_1 varchar, column_2 varchar)");

//使用Statement语句向表中插入数据
stmt.execute("INSERT INTO mytable
values('statementinsert','statementinsert')");

//使用PreparedStatement向表中插入数据
String insertsql ="INSERT INTO mytable values(?,?)";
pstmt =conn.prepareStatement(insertsql);
pstmt.setString(1,"preparedstatementinsert");
pstmt.setString(2,"preparedstatementinsert");
pstmt.executeUpdate();

//使用Statement语句查询数据
ResultSet stmtrs =stmt.executeQuery("SELECT * FROM mytable");

// 处理查询结果
while (stmtrs .next()) {

    // 根据你的表结构，获取并打印字段
    System.out.println(stmtrs .getString("column_1"));
    System.out.println(stmtrs .getString("column_2"));

}

//使用PreparedStatement查询语句
String querysql ="SELECT * FROM mytable where column_1 = ? ";
pstmt=conn.prepareStatement(querysql);
pstmt.setString(1,"preparedstatementinsert");
ResultSet pstmttrs =pstmt.executeQuery();

// 处理查询结果
while (pstmttrs .next()) {

    // 根据你的表结构，获取并打印字段
    System.out.println(pstmttrs .getString("column_1"));
    System.out.println(pstmttrs .getString("column_2"));

}

// 关闭结果集、声明和连接
stmtrs.close();
pstmttrs.close();
pstmt.close();
stmt.close();

```

```

        conn.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

C/C++

使用 C/C++ 连接 TDM 数据库，执行建表、插入数据和查询数据的操作，可以使用 PostgreSQL 的 libpq 库。这是 PostgreSQL 提供用于客户端编程的官方 C 库。TDM 兼容 PostgreSQL。

使用前准备

使用 C/C++ 连接 TDM 数据库前，需要具备以下条件：

- 安装了 PostgreSQL（要求版本 16.0 及以上）和对应的开发库。
- 在 C/C++ 项目中引用 libpq 库。

应用示例

```

#include <stdio.h>#include <stdlib.h>#include <libpq-fe.h>

int main() {

    // 定义连接串， 端口号为TDM_ROUTER的端口号，用户目前仅支持postgres，且默认密码为dameng
    const char* conninfo = "host=localhost port=25432 dbname=postgres user=postgres
password=dameng";

    PGconn* conn = PQconnectdb(conninfo);
    if (PQstatus(conn) == CONNECTION_BAD) {
        fprintf(stderr, "Connection to database failed: %s\n", PQerrorMessage(conn));
        PQfinish(conn);
        return 1;
    }

    // 建表
    PGresult* res = PQexec(conn, "CREATE TABLE IF NOT EXISTS mytable (id int, name
varchar)");

    if (PQresultStatus(res) != PGRES_COMMAND_OK) {
        fprintf(stderr, "CREATE TABLE failed: %s", PQerrorMessage(conn));
        PQclear(res);
        PQfinish(conn);
        return 1;
    }
    PQclear(res);

    // 插入数据
    res = PQexec(conn, "INSERT INTO mytable VALUES (1,'John Doe');");
    if (PQresultStatus(res) != PGRES_COMMAND_OK) {
        fprintf(stderr, "INSERT failed: %s", PQerrorMessage(conn));
        PQclear(res);
        PQfinish(conn);
        return 1;
    }
}

```

```

    }

    PQclear(res);

    // 查询数据
    res = PQexec(conn, "SELECT * FROM mytable;");

    if (PQresultStatus(res) != PGRES_TUPLES_OK) {
        fprintf(stderr, "SELECT failed: %s", PQerrorMessage(conn));
        PQclear(res);
        PQfinish(conn);
        return 1;
    }

    int nFields = PQnfields(res);

    int i;

    for (i = 0; i < nFields; i++) {

        printf("%-15s", PQfname(res, i));

    }

    printf("\n");
    int nTuples = PQntuples(res);
    for (i = 0; i < nTuples; i++) {
        for (int j = 0; j < nFields; j++) {
            printf("%-15s", PQgetvalue(res, i, j));

        }
        printf("\n");
    }

    PQclear(res);
    PQfinish(conn);
    return 0;
}

```

C#

使用前准备

使用 C# 连接 TDM 数据库前，需要具备以下条件：

- 已安装。NET 环境。
- C# 程序中已包含 System.Data.Odbc 包。
- 已经安装了 PostgreSQL 的 ODBC 驱动程序。

应用示例

下面是一个具体的编程实例。

```
using System;using System.Data;using System.Data.Odbc;
```

```

class Program

{
    static void Main()
    {
        // 配置你的ODBC连接字符串，目前用户仅支持postgres，且密码为dameng。

        string connectionString = "Driver={PostgreSQL
Unicode(x64)};Server=localhost;Port=25432;Database=postgres;Username=postgres;Password=d
ameng;";

        // 创建一个表
        ExecuteNonQuery(connectionString, "CREATE TABLE my_table (id int, name
varchar)");

        // 插入数据
        ExecuteNonQuery(connectionString, "INSERT INTO my_table VALUES (1,'John
Doe')");

        // 查询数据
        using (OdbcDataReader reader = ExecuteReader(connectionString, "SELECT * FROM
my_table"))
        {
            while (reader.Read())
            {
                Console.WriteLine(reader["name"]);
                Console.ReadLine();
            }
        }
    }

    static void ExecuteNonQuery(string connectionString, string commandText)
    {
        using (OdbcConnection connection = new OdbcConnection(connectionString))
        {
            using (OdbcCommand command = new OdbcCommand(commandText, connection))
            {
                connection.Open();
                command.ExecuteNonQuery();
            }
        }
    }

    static OdbcDataReader ExecuteReader(string connectionString, string commandText)
    {
        OdbcConnection connection = new OdbcConnection(connectionString);
        OdbcCommand command = new OdbcCommand(commandText, connection);
        connection.Open();
        return command.ExecuteReader();
    }
}

```


Python

使用前准备

使用 Python 连接 TDM 数据库前，需要具备以下条件：

- 已安装 Python 环境及 pip 工具。
- 项目中安装 pyodbc 包。
- 已经安装了 PostgreSQL 的 ODBC 驱动程序。

应用示例

下面用一个具体的编程实例来展示用 Python 通过 ODBC 来访问操作 TDM 数据库。

```
import pyodbc

if __name__ == '__main__':

    # 配置连接字符串
    conn_str = ("DRIVER={PostgreSQL Unicode(x64)};"
               "server=10.14.142.32;"
               "port=25432;"
               "database=postgres;"
               "uid=postgres;"
               "pwd=dameng;")

    # 建立连接
    conn = pyodbc.connect(conn_str, autocommit=True)

    # 创建一个游标对象
    cursor = conn.cursor()

    # 创建表
    cursor.execute("CREATE TABLE mytable (id int, name varchar)")

    # 插入数据
    cursor.execute("INSERT INTO mytable VALUES (1,'Alice')")

    # 提交事务（如果autocommit=False时）
    # conn.commit()

    # 查询数据
    cursor.execute("SELECT * FROM mytable")
    rows = cursor.fetchall()
    for row in rows:
        print(row)

    # 关闭游标和连接
    cursor.close()
    conn.close()
```

Go

使用前准备

使用 Go 连接 TDM 数据库前，需要具备以下条件：

- 已安装 Go 环境。
- 在项目中添加对pq包的引用。

应用示例

下面用一个具体的编程实例来展示如何用 Go语言访问操作 TDM 数据库。

```
package main

import (
    "fmt"
    "log"
)
import "database/sql"
import _ "github.com/lib/pq"

func main() {
    // 连接数据库
    connStr := "host = localhost port=5432 user=postgres dbname=postgres sslmode=disable password=12345678"
    db, err := sql.Open("postgres", connStr)
    if err != nil {
        log.Fatal(err)
    }
    defer db.Close()
    // 创建表
    _, err = db.Exec("CREATE TABLE IF NOT EXISTS goexample3 (id int ,name VARCHAR(255))")
    if err != nil {
        log.Fatal(err)
    }
    // 插入数据
    _, err = db.Exec("INSERT INTO goexample3 values(1,'test')")
    if err != nil {
        log.Fatal(err)
    }
    // 查询数据
    rows, err := db.Query("SELECT * FROM goexample3")
    if err != nil {
        log.Fatal(err)
    }
    defer rows.Close()
    for rows.Next() {
        var id int
        var name string
        if err := rows.Scan(&id, &name); err != nil {
            log.Fatal(err)
        }
        fmt.Printf("ID: %d, Name: %s\n", id, name)
    }
    if err := rows.Err(); err != nil {
        log.Fatal(err)
    }
}
```

时间分区和滚动删除

TDM 支持自定义超表的时间分区，与滚动删除的超时时间。

超表会根据时间自动对时序数据进行分区。每个超表由数据块组成。每个数据块都分配了一个时间范围，并且只包含该范围内的数据。查询时会识别到正确的块进行查询，而不是对整个表进行遍历。

分区查询

查询当前超表的所有数据块列表。

语法格式

```
SELECT SHOW_CHUNKS('table_name');

SELECT * FROM SHOW_CHUNKS('table_name');
```

参数

1. table_name: 要查询的超表。

使用说明

1. SELECT SHOW_CHUNKS('table_name')直接查询所有数据块列表。
2. SELECT * FROM SHOW_CHUNKS('table_name')将其作为SRF调用，能够查询到数据块的具体信息，可对数据块列表按所需条件进行过滤查询。

举例说明

例1. 查询所有数据块列表

```
TDM>SELECT SHOW_CHUNKS('t1');
+-----+
| show_chunks |
+-----+
| cube_0_2025_03_01_00_00_00_chunk |
+-----+
| cube_0_2025_04_01_00_00_00_chunk |
+-----+
```

例2.对数据块列表按所需条件进行过滤查询。.

```
TDM>SELECT name,cube_id,range_end FROM SHOW_CHUNKS('t1') WHERE range_end > '2025-03-08
00:00:00';
+-----+-----+-----+
| name | cube_id | range_end |
+-----+-----+-----+
| cube_0_2025_043_01_00_00_00_chunk | 0 | 2025-04-31 23:59:50.000000 |
+-----+-----+-----+
```

作为 SRF 查询时，可查询到如下信息:name:数据块名称,range_start:数据块中最小时间戳, range_end:数据块中最小时间戳。

自定义时间分区

TDM支持在建表时指定时间分区，将存储间隔修改为N天、N月、N年。具体语法参考[定义时序表](#)。

举例说明

1. 创建关于天气的weather表，且指定时间分区为一个月。

```
TDM>CREATE HYPERTABLE weather(location string as dimension,temperature integer as
measure,`time` timestamp) partition_interval 1 month;
Successfully create table
```

滚动删除

删除时间范围完全超过指定过期时间的数据块，会将完全过期的数据块进行删除，并返回删除数据块的列表。滚动删除是一种为方便运维而设计的机制，因此它不保证删除的实时性和准确性。如需精准控制删除操作，请使用DELETE语句。

语法格式

```
SELECT DROP_CHUNKS('table_name', interval_val);
```

参数

1. table_name: 要更新时间分区的超表。
2. Interval_val: 更新的时间分区，格式为：interval ‘interval数值’ 单位 或 interval ‘interval数值 单位’。目前单位只支持day、month、year。当单位缺省时，默认单位为day。interval数值必须为整数

举例说明

1. 删除时间块，过期时间是一年

```
TDM>SELECT DROP_CHUNKS('t1', interval '1' year);
+-----+
| drop_chunks |
+-----+
| cube_0_2024_03_01_00_00_00_chunk |
+-----+
| cube_0_2025_04_01_00_00_00_chunk |
+-----+
```

2. 删除时间块，过期时间10，使用缺省单位

```
TDM>SELECT DROP_CHUNKS('t1', interval '10');
+-----+
| drop_chunks |
+-----+
| cube_0_2024_03_01_00_00_00_chunk |
+-----+
| cube_0_2025_04_01_00_00_00_chunk |
+-----+
```

数据保留

TDM 时序数据库针对过期数据清理提供两类能力：支持用户手动执行语句删除指定分区数据，同时内置自动过期数据清理能力。用户可通过 ADD_RETENTION_POLICY 为时序超表配置数据保留策略，系统将按照设定周期自动扫描、分批删除超出保留时长的历史分区，无需人工干预，实现存储资源自动回收，降低海量时序数据的存储运维成本。

创建保留策略

语法格式：

```
SELECT ADD_RETENTION_POLICY ('table_name', interval,if_not_exists );
```

参数

1. table_name: 字符类型，必填。指定需要配置数据保留策略的时序超表名称。
2. Interval:时间间隔类型，必填。定义数据最长保留时长，超过该时长的数据块会被系统自动定时清理。支持两种标准书写格式，语法完全等效，单位只支持day，month，year。
 - a. 格式 1: interval '1 day'（数值与单位合并写在单引号内） 示例: interval '7 day'、interval '1 month'、interval '1 year'。
 - b. 格式 2: interval '1' day（数值、单位拆分，数值单独包裹引号） 示例: interval '7' day、interval '3' month、interval '1' year。
3. if_not_exists:布尔类型，可选。设置为true时，目标表已存在保留策略时，执行不抛出错误； 设置为 false时，目标表已存在保留策略时，直接返回报错； 不传入该参数时，默认值为 false。

使用说明

1. 配置保留策略后，系统默认每 5 分钟执行一次后台清理任务，扫描并自动删除超出保留时长的数据块；清理不会一次性删除全部过期数据文件，完整清理流程通常需要数分钟完成，清理为后台清理，不影响数据库正常提供服务。
2. 过期数据清理任务的执行间隔支持自定义调整，修改配置文件 config/base_config 中保留策略相关参数（ts_retention_check_timeout_sec）即可生效。
3. 当前数据库采用行列混合存储架构，定时清理任务执行后，可能存在少量行存残留数据未即时清理；待该部分数据完成列存合并后，会在下一轮定时清理任务中被自动清除。

举例说明

例1. 为weather表添加保留策略，保留数据为6个月。

```
TDM>SELECT ADD_RETENTION_POLICY('weather',interval '6' month);
+-----+
| add_retention_policy|
+-----+
| 0                  |
+-----+
```

移除保留策略

REMOVE_RETENTION_POLICY 用于移除时序超表已绑定的数据自动保留清理策略，策略删除后系统将不再定时自动清理该表的过期分区，所有历史时序数据会永久留存，如需清理过期数据只能通过手动分区删除函数操作。

语法格式

```
SELECT REMOVE_RETENTION_POLICY('table_name',if_exists);
```

参数

1. table_name: 字符类型，必填。指定需要配置数据保留策略的时序超表名称。
2. if_exists: 布尔类型，可选。设置为true时，目标表未配置策略时，执行不抛出错误；设置为 false 时，目标表不存在保留策略时，直接返回报错；不传入该参数时，默认值为 false。

举例说明

例1. 移除weather表的保留策略。

```
TDM>SELECT REMOVE_RETENTION_POLICY('weather');
+-----+
| remove_retention_policy |
+-----+
| NULL                    |
+-----+
```

查询保留策略

SHOW_RETENTION_POLICY 用于查询指定时序超表已配置的数据自动保留清理策略，可获取策略保留时长信息。若目标表未配置任何保留策略，查询返回disabled。

语法格式

```
SELECT SHOW_RETENTION_POLICY('table_name');
```

举例说明

例1. 查询weather表的保留策略。

```
TDM>SELECT SHOW_RETENTION_POLICY('weather');
+-----+
| show_retention_policy |
+-----+
| 6 month               |
+-----+
```

数据库角色

TDM使用角色的概念来管理数据库访问权限，一个角色可以被看成是一个数据库用户或者是一个数据库用户组。postgres是默认的超级用户。

基本概念

角色

“角色”是数据库用于管理访问控制权限的一种抽象。

- 一个“角色”既可以被看作是数据库用户，也可以被看做用户组，如果一个角色被允许登录（LOGIN 属性），那么它就可以当作“用户”使用，否则只能当作“用户组”使用，用来集中管理权限，然后让其他可登录的角色（用户）加入它。
- “角色”可能拥有数据库中某些对象上的权限，他也可以将这些权限赋给其他“角色”，用于对这些对象进行访问。
- “角色”之间可以构成“成员关系”，并且这种关系是可以传递的。
- “角色”是属于整个数据库集簇的，而非存在于单个数据库内。

角色属性

数据库角色可以具有多个属性，定义其特权并与客户端身份验证系统交互。如下表所示：

属性	属性描述
login privilege	具有此属性的“角色”可作为建立数据库连接时的初始用户。
superuser status	具有此属性的“角色”被视为“超级用户”，超级用户可以绕过所有权限检查，除了登录检查。
database creation	具有此属性的“角色”可以创建创建数据库。
role creation	具有此属性的“角色”可以创建其他“角色”，但是不能创建“超级用户”。
initiating replication	具有此属性的“角色”可以执行备份，TDM目前不支持设置该属性。
password	此属性为“角色”的数据库登录密码。
Inheritance of privileges	此属性表示权限的继承性。默认情况下，如果一个“角色”是另一个“角色”的组成员，那么可以继承其权限。
bypassing row-level security	具有此属性的“角色”可以绕过行级安全策略，TDM目前不支持设置该属性。
connection limit	此属性用于控制“角色”可并发建立的连接数量，值-1表示无限制。

角色组

TDM将角色分组来简化权限的管理。可以对一组角色进行赋权和回收权限。当某个“角色”之下具有其他成员时，这个“角色”就成为了“组角色”。此时，该“角色”既是单一“角色”，又是“组角色”。

权限分类

创建角色后，可以为角色赋予制定权限，包括的权限种类如下表所示：

权限	缩写	相关对象
SELECT	r (“read”)	表、列、序列
INSERT	a (“append”)	表、列
UPDATE	w (“write”)	表、列、序列
DELETE	d	表
TRUNCATE	D	表
CREATE	C	数据库、模式、表空间（TDM暂无该概念）
CONNECT	c	数据库
USAGE	U	模式、序列、类型

数据库对象权限如下表所示：

对象类型	所有权限
数据库	CTc
函数	X
模式	UC
表	arwdDxt
列	arwx

创建角色

[CREATE ROLE] 和 [CREATE GROUP] 定义一个新的数据库角色。也可以使用别名[CREATE USER]。

语法格式：

```
CREATE ROLE name [ [ WITH ] option [ ... ] ]
```

这里的option可以是：

```
SUPERUSER | NOSUPERUSER

| CREATEDB | NOCREATEDB

| CREATEROLE | NOCREATEROLE

| INHERIT | NOINHERIT

| LOGIN | NOLOGIN

| CONNECTION LIMIT connlimit

| [ ENCRYPTED ] PASSWORD 'password' | PASSWORD NULL

| VALID UNTIL 'timestamp'
```



```
| IN ROLE role_name [, ...]

| IN GROUP role_name [, ...]

| ROLE role_name [, ...]

| ADMIN role_name [, ...]

| USER role_name [, ...]

| SYSID uid
```

CREATE ROLE语句会在数据库内创建一个"角色"。想要执行CREATE ROLE命令，需要具有CREATE ROLE权限的用户，或是数据库超级用户。

CREATE GROUP 是 CREATE ROLE 的同义词。

CREATE USER 是CREATE ROLE 的别名，主要区别为使用 CREATE USER 默认属性为LOGIN,而CREATE ROLE默认为NOLOGIN.

参数

1. name: 新角色的名称
2. SUPERUSER|NOSUPERUSER:此选项表示是否创建超级用户。只有超级用户才能创建超级用户。默认值为NOSUPERUSER。
3. CREATEDB | NOCREATEDB:此选项定义了被创建的“角色”是否具有创建数据库的权限。只有超级用户和具有CREATEDB权限的“角色”才能指定CREATEDB选项。默认为NOCREATEDB。
4. CREATEROLE | NOCREATEROLE:此选项定义了被创建的“角色”是否具有对其他“角色”执行create、drop、alter的能力。默认值为NOCREATEROLE。
5. INHERIT | NOINHERIT:此选项定义了当一个“角色”作为成员加入到另一个“角色”中时，成员的继承状态。在GRANT命令中将此“角色”作为成员加入“组”时，此选项也是作为默认的继承状态。默认值为INHERIT。
6. LOGIN|NOLOGIN:此选项定义了“角色”是否具有登录数据库的权限。也就是说，“角色”是否可以在客户端认证过程中，被指定为初始化会话连接的用户名。默认值为NOLOGIN，使用CREATE USER 默认属性为LOGIN。
7. CONNECTION LIMIT connlimit: 如果一个“角色”具有·LOGIN·属性，此选项定义了“角色”可使用的并发连接数量。默认值为-1，即无限制。
8. [ENCRYPTED] PASSWORD 'password' PASSWORD NULL:

此选项定义了“角色”登录时使用的密码。密码必须包含小写字母、大写字母、数字和特殊符号。如果没有设置密码，则密码值会被设置为null，并且进行密码登录验证时必定会失败。null密码也可以使用PASSWORD NULL显式指定。密码在保存时始终是加密的，因此 ENCRYPTED 选项没有实际的作用，此选项仅用于兼容性。注意，不要使用空字符串作为密码，空字符串会被视为 null，但是在某些场景下容易产生歧义。此外密码长度至少8字符，需要包含大小写字母、数字、特殊字符。

1. VALID UNTIL 'timestamp':此选项定义了“角色”的密码有效期截止时间。如果没有设置此选项，则永久有效。
2. IN ROLE role_name:此选项用于将新创建的“角色”作为成员加入到一个或者多个已有的“角色”中，（注意没有选项可以把新角色作为一个管理员加入，需要用单独的GRANT命令来完成）。
3. IN GROUP role_name:此选项为IN ROLE的旧式写法。
4. ROLE role_name:此选项将一个或多个已有“角色”作为成员加入进来，这会将新创建的“角色”变为“组角色”。
5. ADMIN role_name:此选项与ROLE的效果相同，但是被提及的“角色”被使用ADMIN选项加入到新角色中，这使得他们具有将成员关系赋予其他“角色”的能力。
6. USER role_name:此选项为ROLE role_name的旧式写法。
7. SYSID uid:此选项仅用于向后兼容。

使用说明

1. CREATE ROLE 语句会在数据库内创建一个“角色”。想要执行 CREATE ROLE 命令，需要具有 CREATE ROLE 权限，或是数据库超级用户。
2. 如果设置了 CONNECTION LIMIT，在仅剩一个连接可用时，同时发起的两个连接都可能失败。CONNECTIONLIMIT不适用于超级用户。

举例说明

例1：创建一个具有登录能力但是没有口令的角色，实际登录时该角色因为没有口令无法登录。

```
TDM>CREATE ROLE user1 LOGIN;  
  
Successfully create role
```

例2：创建一个有口令的角色。

```
TDM>CREATE ROLE user2 WITH LOGIN PASSWORD '12ab#$CD';  
  
Successfully create role
```

例3：创建一个有口令的角色,且口令有效期至2025年1月1日。

```
TDM>CREATE ROLE user3 WITH LOGIN PASSWORD '12ab#$CD' VALID UNTIL '2025-01-01';  
  
Successfully create role
```

sql 例4：创建一个能够创建数据库并且管理角色的角色。

```
TDM>CREATE ROLE adminrole WITH CREATEDB CREATEROLE;  
  
Successfully create role
```

删除角色

通过该语句可以删除多个“角色”。

语法格式：

```
DROP ROLE [ IF EXISTS ] name [, ...]
```

参数：

1. IF EXISTS：如果“角色”不存在，不会报错，而是给出提示信息。
2. Name：想要移除的“角色”名称。

使用说明

1. 如果要删除“超级用户”，自身必须也是“超级用户”。
2. 如果要删除“一般角色”，必须具有 CREATEROL 权限，并且被赋予了 ADMIN OPTION。
3. 如果“角色”被数据库内其他对象关联，则无法被删除。因此必须先删除或转移“角色”拥有的所有对象，并且移除“角色”被赋予的其他对象上的权限。
4. DROP ROLE会自动移除当前“角色”和其他“角色”之间的成员关系。

举例说明

例1：移除一个能登录但是无口令的角色。

```
TDM>DROP ROLE user1 ;
```

```
Successfully drop role
```

修改角色

修改角色属性

ALTER ROLE语句用于修改“角色”的属性，但是此语句不能修改“角色”之间的成员关系。

语法格式：

```
ALTER ROLE role_specification [ WITH ] option [ ... ]
```

其中option 可以是：

```
SUPERUSER | NOSUPERUSER

| CREATEDB | NOCREATEDB

| CREATEROLE | NOCREATEROLE

| INHERIT | NOINHERIT

| LOGIN | NOLOGIN

| CONNECTION LIMIT connlimit

| [ ENCRYPTED ] PASSWORD 'password' | PASSWORD NULL

| VALID UNTIL 'timestamp'
```

其中role_specification 可以是：

```
role_name

| CURRENT_ROLE

| CURRENT_USER

| SESSION_USER
```

使用说明：

1. 此变体可以修改大多数在CREATE ROLE中指定的属性（成员关系需要使用GRANT和REVOKE来处理）。
2. 没有在命令中提及的属性会保持不变。
3. “超级用户”可以修改任何“角色”的任何属性。
4. “普通角色”无法修改“超级用户”的属性。
5. “普通角色”如果拥有其他“角色”的ADMIN OPTION，则可以修改其大多数属性。
6. “普通角色”如果想要修改CREATEDB属性，则自身必须具有相应的属性。
7. 不具备特殊权限的“角色”只能修改自身的密码。

举例说明

例1：更改一个角色的口令。

```
TDM>ALTER ROLE user1 WITH PASSWORD 'new12ab#$CD';
```

Successfully alter role

例2：移除一个角色的口令。

```
TDM>ALTER ROLE user1 WITH PASSWORD NULL;
```

Successfully alter role

修改角色名

ALTER ROLE语句用于修改角色名

语法格式：

```
ALTER ROLE name RENAME TO new_name
```

参数：

1. name：被修改属性的“角色”名称
2. new_name：“角色”的新名称。

使用说明：

1. 此变体用于修改“角色”名称。
2. “超级用户”可以修改任何“角色”的名称。
3. 具有CREATEROLE权限的“普通角色”，如果同时拥有其他“角色”上的ADMIN OPTION属性，则可以修改其名称。
4. 建立当前会话的“角色”的名称无法被修改。
5. 重命名一个“角色”会导致使用MD5加密的密码被清空。

举例说明

例1：修改角色的名称。

```
TDM>ALTER ROLE user1 RENAME TO user1new;
```

Successfully alter role

修改角色组

ALTER GROUP语句是一个过时的命令。主要用于向后兼容，用于修改“角色”之间的成员关系。

语法格式：

```
ALTER GROUP role_specification ADD USER user_name[,...]
```

```
ALTER GROUP role_specification DROP USER user_name[,...]
```

赋予权限

通过GRANT命令来定义数据库对象的访问特权。GRANT 命令包含两种变体，一种用于赋予数据库对象上的权限，另一种用于赋予“成员关系”。

赋予数据库对象权限

语法格式：

```
GRANT { { SELECT | INSERT | UPDATE | DELETE | TRUNCATE }[, ...] | ALL [ PRIVILEGES ] }  
  
    ON { [ TABLE ] table_name [, ...]  
  
        | ALL TABLES IN SCHEMA schema_name [, ...] }  
  
    TO role_specification [, ...] [ WITH GRANT OPTION ]  
  
    [ GRANTED BY role_specification ]  
  
  
GRANT { { SELECT | INSERT | UPDATE } ( column_name [, ...] )  
  
[, ...] | ALL [ PRIVILEGES ] ( column_name [, ...] ) }  
  
    ON [ TABLE ] table_name [, ...]  
  
    TO role_specification [, ...] [ WITH GRANT OPTION ]  
  
    [ GRANTED BY role_specification ]  
  
  
GRANT { { CREATE | CONNECT } [, ...] | ALL [ PRIVILEGES ] }  
  
    ON DATABASE database_name [, ...]  
  
    TO role_specification [, ...] [ WITH GRANT OPTION ]  
  
    [ GRANTED BY role_specification ]  
  
  
GRANT { { CREATE | USAGE } [, ...] | ALL [ PRIVILEGES ] }  
  
    ON SCHEMA schema_name [, ...]  
  
    TO role_specification [, ...] [ WITH GRANT OPTION ]  
  
    [ GRANTED BY role_specification ]
```

其中role_specification 可以是：

```
[ GROUP ] role_name  
  
| PUBLIC
```

| CURRENT_ROLE

| CURRENT_USER

| SESSION_USER

使用说明:

1. 以上用法将数据库对象的指定特权赋予给角色。
2. 如果指定了with grant option,则特权的接受者可以接着把这个权限授予其他人。如果没有授权选项,接收者就不能把权限再授予其他人。
3. 如果指定了GRANTED BY,那么GRANT操作会被记录为是由一个特定的“角色”所执行的。被指定为grantor的“角色”必须具有“目标角色”的ADMIN OPTION,或是数据库初始化用户。如果被指定为grantor的“角色”不是数据库初始化用户,那么grant操作依赖于ADMIN OPTION的“持续性”。如果grantor的ADMIN OPTION被REVOKE,则GRANT操作也会被撤回。
4. 当超级用户对某个对象执行GRANT时,相当于是对象的拥有者在执行此操作。
5. TO PUBLIC: 则会将权限赋予当前所有角色。

举例说明:

例1: 把表films上的插入特权授予给所用用户。

```
TDM>CREATE HYPERTABLE films (col int as measure,ts timestamp);

Successfully create table

TDM>GRANT INSERT ON films TO PUBLIC;

GRANT
```

赋予成员关系

语法格式

```
GRANT role_name [, ...] TO role_specification [, ...]

[ WITH ADMIN OPTION ]

[ GRANTED BY role_specification ]
```

其中role_specification 可以是:

[GROUP] role_name

| CURRENT_ROLE

| CURRENT_USER

| SESSION_USER

使用说明:

1. 以上用法将一个“角色”加入到一个“组角色”中,构成“成员关系”。
2. 如果指定了WITH ADMIN OPTION,成员接着可以把该角色的成员关系授予其他用户即把其他角色加入该组,也可以撤回该角色的成员关系即把已有成员从组中移除。“普通角色”不具有其自身的

ADMIN OPTION。

3. 如果指定了GRANTED BY，那么GRANT操作会被记录为是由一个特定的“角色”所执行的。被指定为grantor的“角色”必须具有“目标角色”的ADMIN OPTION，或是数据库初始化用户。如果被指定为grantor的“角色”不是数据库初始化用户，那么grant操作依赖于ADMIN OPTION的“持续性”。如果grantor的ADMIN OPTION被REVOKE，则GRANT操作也会被撤回。

举例说明：

例1：把角色app_user中的权限授予给用户app_admin。

```
TDM>CREATE ROLE app_user;

CREATE ROLE

TDM>CREATE ROLE app_admin;

CREATE ROLE

TDM>GRANT app_user TO app_admin;

GRANT ROLE
```

回收权限

通过REVOKE命令来对权限进行回收，和角色成员关系移除。

回收数据库对象权限

语法格式：

```
REVOKE [ GRANT OPTION FOR ]

    { { SELECT | INSERT | UPDATE | DELETE | TRUNCATE } [, ...] | ALL [ PRIVILEGES ] }

ON { [ TABLE ] table_name [, ...]

    | ALL TABLES IN SCHEMA schema_name [, ...] }

FROM role_specification [, ...]

[ GRANTED BY role_specification ]

[ CASCADE | RESTRICT ]

REVOKE [ GRANT OPTION FOR ]

    { { SELECT | INSERT | UPDATE } ( column_name [, ...] )[, ...] | ALL [ PRIVILEGES ] (

column_name [, ...] ) }

ON [ TABLE ] table_name [, ...]

FROM role_specification [, ...]

[ GRANTED BY role_specification ]
```

```
[ CASCADE | RESTRICT ]
```

```
REVOKE [ GRANT OPTION FOR ]
```

```
{ { CREATE | CONNECT } [, ...] | ALL [ PRIVILEGES ] }
```

```
ON DATABASE database_name [, ...]
```

```
FROM role_specification [, ...]
```

```
[ GRANTED BY role_specification ]
```

```
[ CASCADE | RESTRICT ]
```

```
REVOKE [ GRANT OPTION FOR ]
```

```
{ { CREATE | USAGE } [, ...] | ALL [ PRIVILEGES ] }
```

```
ON SCHEMA schema_name [, ...]
```

```
FROM role_specification [, ...]
```

```
[ GRANTED BY role_specification ]
```

```
[ CASCADE | RESTRICT ]
```

其中role_specification 可以是：

```
[ GROUP ] role_name
```

```
| PUBLIC
```

```
| CURRENT_ROLE
```

```
| CURRENT_USER
```

```
| SESSION_USER
```

使用说明：

1. 以上用法将数据库对象的指定特权从角色回收。
2. 用户只能移除由他们直接分配出去的权限。例如，A使用GRANT OPTION 将权限分配给B，B又将权限分配给C，那么A无法直接从C处回收权限。但是A可以在回收B权限时使用CASCADE，则C的权限也会被回收。
3. RESTRICT是默认行为，即不进行级联处理。
4. 如果“角色”并非对象的拥有者，那么“角色”在对象上执行权限回收时，只有此“角色”具有GRANT OPTION的权限会被回收。
5. 如果某个“角色A”是对象的所有者的成员，或是在对象上拥有GRANT OPTION选项的“角色”的成员，则A也可以在对象上执行权限回收。
6. 如果“角色”具有GRANT OPTION，则只有GRANT OPTION被移除，权限本身并不会被移除；否则，权限会被移除。

举例说明：

例1：从public收回表films上的插入特权。

```
TDM>REVOKE INSERT ON films FROM PUBLIC;
```

```
REVOKE
```

回收成员关系

语法格式：

```
REVOKE [ { ADMIN | INHERIT | SET } OPTION FOR ]  
  
      role_name [, ...] FROM role_specification [, ...]  
  
      [ GRANTED BY role_specification ]  
  
      [ CASCADE | RESTRICT ]
```

其中role_specification 可以是：

```
[ GROUP ] role_name  
  
| PUBLIC  
  
| CURRENT_ROLE  
  
| CURRENT_USER  
  
| SESSION_USER
```

使用说明：

1. 以上REVOKE变体可以回收一个角色中的成员关系。
2. RESTRICT是默认行为，即不进行级联处理。

举例说明：

例1：从用户app_admin收回角色app_user中的成员关系。

```
TDM>REVOKE app_user FROM app_admin;
```

```
REVOKE ROLE
```

服务器配置

TDM默认是以列存储的方式存储数据的，无需手动配置。其他配置项需要根据实际使用需求配置。

配置文件位置

TDM服务配置文件均放在安装包tdm_XXX的config文件夹中详细参照安装包目录及配置文件介绍。

base_cfg.ini为数据库服务配置文件。

主要配置数据库服务的日志文件路径、数据文件路径、时区、许可证、线程数、存储加密/不加密

cluster_cfg.ini 是集群配置文件。

这样配置数据库服务engine端口、rpc端口范围、部署服务器地址、数据存储分片（cube）、副本数等。

install_cfg.ini 是安装配置文件。

主要配置安装路径、远程安装用户名和密码、ssh端口。

pg/postgresql.conf是连接设置的配置文件。

主要配置服务器监听的TCP端口、最大连接数、传输加密SSL相关的配置。

连接设置

修改**pg/postgresql.conf**。

1. port：服务器监听的TCP端口、默认16502。

注意：分布式集群，所有节点服务器用同一个端口号。

2. max_connections 决定数据库的最大并发连接数。默认值通常是 100 个连接。

注意：以上2个参数在安装部署启动时设置、已经运行的数据服务要变更该参数，请参照修改数据库参数步骤。

传输加密SSL

修改**pg/postgresql.conf**。

1. ssl = on 启用ssl连接。

说明：这个参数只能在postgresql.conf文件中设置。默认是被注释掉的，即未启用ssl连接。

2. ssl_cert_file='server.crt'

说明：指定包含 SSL 服务器证书的文件名。相对路径是相对config/pg的和postgresql.conf配置文件同级。这个参数只能在postgresql.conf文件中设置。默认值是注释掉的、在启用ssl连接后开启。

3. ssl_key_file = 'server.key'

说明：指定包含 SSL 服务器私钥的文件名。相对路径是相对config/pg的和postgresql.conf配置文件同级。这个参数只能在postgresql.conf文件中设置。默认值是注释掉的、在启用ssl连接后开启。

注意：以上3个参数在安装部署启动时设置、已经运行的数据库服务要变更该参数，请参照修改数据库参数步骤。

存储加密

修改base_cfg.ini。

1. encrypt_type 配置是否开启存储加密。

说明：默认值plain（非存储加密）、AES_128_CBC（存储加密）。

注意：该参数在安装部署启动时设置、已经运行的数据库服务要变更该参数，请参照修改数据库参数步骤。

结果集缓存

修改base_cfg.ini。

1. query_cache_enabled 配置是否开启结果集缓存功能。
2. query_cache_row_limit 配置缓存结果集的最大行数限制。

说明：配置query_cache_enabled=true 开启数据库结果集缓存功能，表缓存设置才可以生效，query_cache_row_limit默认值1000，根据内存调整。

注意：该参数在安装部署启动时设置、已经运行的数据库服务要变更该参数，请参照修改数据库参数步骤。

附录1 关键字

本小节列出了TDM中作为关键字的所有记号。TDM区分保留关键字和非保留关键字。根据SQL标准，保留关键字才是真正的关键字，不允许使用保留关键字作为标识符。非保留关键字仅仅是在特定上下文中具有特殊的含义并且可以在其他上下文中被用作标识符。大部分非保留关键字实际上是SQL指定的内建表和内建函数的名字。以下不带 *号* 的为*非保留关键字*，带*号* 的为保留关键字。

A

ABORT、ABSOLUTE、ACCESS、ACTION、ADD、ADMIN、AFTER、AGGREGATE、ALL、*ALSO*、*ALTER*、*ALWAYS*、*ANALYSE*、ANALYZE、*AND*、ANY、*ARRAY*、AS、*ASC*、ASENSITIVE、ASSERTION、ASSIGNMENT、ASYMMETRIC、*AT*、*ATOMIC*、*ATTACH*、*ATTRIBUTE*、*AUTHORIZATION*

B

BACKWARD、BEFORE、BEGIN、BETWEEN、BIGINT、BINARY、*BIT*、*BOOLEAN*、*BOTH*、BREADTH、BY

C

CACHE、CALL、CALLED、CASCADE、CASCADED、CASE、*CAST*、CATALOG、CHAIN、CHAR、CHARACTER、CHARACTERISTICS、CHECK、*CHECKPOINT*、*CLASS*、*CLOSE*、*CLUSTER*、*COALESCE*、*COLLATE*、COLLATION、*COLUMN*、COLUMNS、COMMENT、COMMENTS、COMMIT、COMMITTED、COMPRESSION、CONCURRENTLY、*CONFIGURATION*、*CONFLICT*、*CONNECTION*、*CONSTRAINT*、CONSTRAINTS、CONTENT、CONTINUE、CONVERSION、COPY、COST、CREATE、*CROSS*、CSV、CUBE、CURRENT、CURRENT_CATALOG、*CURRENT_DATE*、CURRENT_ROLE、*CURRENT_SCHEMA*、CURRENT_TIME、CURRENT_TIMESTAMP、CURRENT_USER*、CYCLECURSOR

D

DAY、DEALLOCATE、DEC、DECIMAL、DECLARE、DEFAULT、*DEFAULTS*、*DEFERRABLE*、DEFERRED、DEFINER、DELETE、DELIMITER、DELIMITERS、DEPENDS、DEPTH、DESC、*DETACH*、*DICTIONARY*、*DISABLE*、*DISCARD*、*DISTINCT*、*DO*、DOCUMENT、DOMAIN、DOUBLE、DROP

E

EACH、ELSE、*ENABLE*、*ENCODING*、*ENCRYPTED*、*END*、ENUM、ESCAPE、EVENT、EXCEPT*、EXCLUDE、EXCLUDING、EXCLUSIVE、EXECUTE、EXISTS、EXPLAIN、EXPRESSION、EXTENSION、EXTERNAL、EXTRACT

F

FALSE*、FAMILY、FETCH、FILTER、FINALIZE、FIRST、FLOAT、FOLLOWING、FOR、*FORCE*、*FOREIGN*、FORWARD、FREEZE、*FROM*、*FULL*、FUNCTION、FUNCTIONS

G

GENERATED、GLOBAL、GRANT、*GRANTED*、*GREATEST*、*GROUP*、GROUPING、GROUPS

H

HANDLER、HAVING*、HEADER、HOLD、HOUR

I

IDENTITY, IF, ILIKE, *IMMEDIATE*, *IMMUTABLE*, *IMPLICIT*, *IMPORT*, *IN*, INCLUDE, INCLUDING, INCREMENT, INDEX, INDEXES, INHERIT, INHERITS, INITIALLY, *INLINE*, *INNER*, INOUT, INPUT, INSENSITIVE, INSERT, INSTEAD, INT, INTEGER, INTERSECT, *INTERVAL*, *INTO*, INVOKER, IS, *ISNULL*, *ISOLATION*

J

JOIN*

K

KEY

L

LABEL, LANGUAGE, LARGE, LAS, LATERAL, *LEAD*, *LEADING*, LEAKPROOF, LEAST, LEFT, *LEVEL*, *LIKE*, LIMIT, *LISTEN*, *LOAD*, *LOCAL*, *LOCALTIME*, LOCALTIMESTAMP*, LOCATION, LOCK, LOCKED, LOGGED

M

MAPPING, MATCH, MATERIALIZED, MAXVALUE, METHOD, MINUTE, MINVALUE, MODE, MONTH, MOVE

N

NAME, NAMES, NATIONAL, NATURAL, *NCHAR*, *NEW*, *NEXT*, *NFC*, *NFD*, *NFKC*, *NFKD*, *NO*, *NONE*, *NORMALIZE*, *NORMALIZED*, *NOT*, NOTHING, NOTIFY, NOTNULL, *NOWAIT*, *NULL*, NULLIF, NULLS, NUMERIC

O

OBJECT, OF, OFF, OFFSET, *OID*s, *OLD*, *ON*, *ONLY*, OPERATOR, OPTION, OPTIONS, OR, *ORDER*, ORDINALITY, OTHERS, OUT, OUTER, *OVER*, *OVERLAPS*, OVERLAY, OVERRIDING, OWNED, OWNER

P

PARALLEL, PARSER, PARTIAL, PARTITION, PASSING, PASSWORD, PLACING, *PLANS*, *POLICY*, *POSITION*, *PRECEDING*, *PRECISION*, *PREPARE*, *PREPARED*, *PRESERVE*, *PRIMARY*, PRIOR, PRIVILEGES, PROCEDURAL, PROCEDURE, PROCEDURES, PROGRAM, PUBLICATION

Q

QUOTE

R

RANGE, READ, REAL, REASSIGN, RECHECK, RECURSIVE, REF, REFERENCES, *REFERENCING*, *REFRESH*, *REINDEX*, *RELATIVE*, *RELEASE*, *RENAME*, *REPEATABLE*, *REPLACE*, *REPLICA*, *RESET*, *RESTART*, *RESTRICT*, *RETURN*, *RETURNING*, RETURNS, REVOKE, RIGHT*, ROLE, ROLLBACK, ROLLUP, ROUTINE, ROUTINES, ROW, ROWS, RULE

S

SAVEPOINT, SCHEMA, SCHEMAS, SCROLL, SEARCH, SECOND, SECURITY, SELECT*, SEQUENCE, SEQUENCES, SERIALIZABLE, SERVER, SESSION, SESSION_USER, *SET*, *SETOF*, *SETS*, *SHARE*, *SHOW*, *SIMILAR*, SIMPLE, SKIP, SMALLINT, SNAPSHOT, SOME, *SQL*, *STABLE*, *STANDALONE*, *START*, *STATEMENT*, *STATISTICS*, *STDIN*, *STDOUT*, *STORAGE*, *STORED*, *STRICT*, *STRIP*, *SUBSCRIPTION*, *SUBSTRING*, *SUPPORT*, *SYMMETRIC*, SYSID, SYSTEM

T

TABLE, *TABLE*, *TABLESAMPLE*, TABLESPACE, TEMP, TEMPLATE, TEMPORARY, TEXT, THEN, *TIES*, *TIME*, *TIMESTAMP*, *TO*, *TRAILING*, TRANSACTION, TRANSFORM, TREAT, TRIGGER, TRIM, TRUE, TRUNCATE, TRUSTED, TYPE*, TYPES

U

UESCAPE, UNBOUNDED, UNCOMMITTED, UNENCRYPTED, UNION, *UNIQUE*, UNKNOWN, UNLISTEN, UNLOGGED, UNTIL, UPDATE, USER, *USING*

V

VACUUM, VALID, VALIDATE, VALIDATOR, VALUE, VALUES, VARCHAR, VARIADIC, *VARYING*, *VERBOSE*, VERSION, VIEW, VIEWS, VOLATILE

W

WHEN, *WHERE*, WHITESPACE, WINDOW, *WITH*, WITHIN, WITHOUT, WORK, WRAPPER, WRITE

X

XML, XMLATTRIBUTES, XMLCONCAT, XMLELEMENT, XMLEXISTS, XMLFOREST, XMLNAMESPACES, XMLPARSE, XMLPI, XMLROOT, XMLSERIALIZE, XMLTABLE

Y

YEAR, YES

Z

ZONE

附录2 SQL语法描述说明

|| 该符号表示或者，该符号限定的语法选项在实际的语句中只能出现一个。

{ } 该部分必须出现，大括号指明大括号内的语法选项在实际的语句中可以出现 0...N 次（N 为大于 0 的自然数），但是大括号本身不能出现在语句中。

[] 可选部分，中括号指明中括号内的语法选项在实际的语句中可以出现 0...1 次，但是中括号本身不能出现在语句中。

... 该符号表示它前面的元素可以被重复。

关键字 关键字在 TDM_SQL 语言中具有特殊意义，在 SQL 语法描述中，关键字以大写形式出现。但在实际书写 SQL 语句时，关键字可以为大写也可以为小写。

附录3 达梦时序数据库技术支持

如果您在安装或使用达梦时序数据库时出现了问题，您可以通过以下途径与我们联系，我们的技术支持工程师会为您提供服务。

武汉达梦数据库股份有限公司

地址：武汉市东湖新技术开发区甲铺岭街39号达梦中国数据库产业基地

邮编：430073

电话：(+86)027-87588000

传真：(+86)027-87588810

北京达梦数据库技术有限公司

地址：北京市海淀区中关村南大街2号数码大厦B座1003

邮编：100086

电话：(+86)010-51727900

上海达梦数据库有限公司

地址：上海市浦东张江高科技园区博霞路50号201室

邮编：201203

电话：(+86) 021-33932717

传真：(+86) 021-33932717-801

四川蜀天梦图数据科技有限公司

地址：成都市天府新区湖畔西路99号B7栋（天府英才中心）6层

电话：(+86)028-61458838

达梦数据技术（江苏）有限公司

地址：江苏省苏州市吴中区吴中大道1421号越旺智慧谷B区B2栋16楼

邮编：215100

电话：0512-65285955

传真：0512-65286955

技术服务：

电话：400-991-6599

邮箱：dmtech@dameng.com